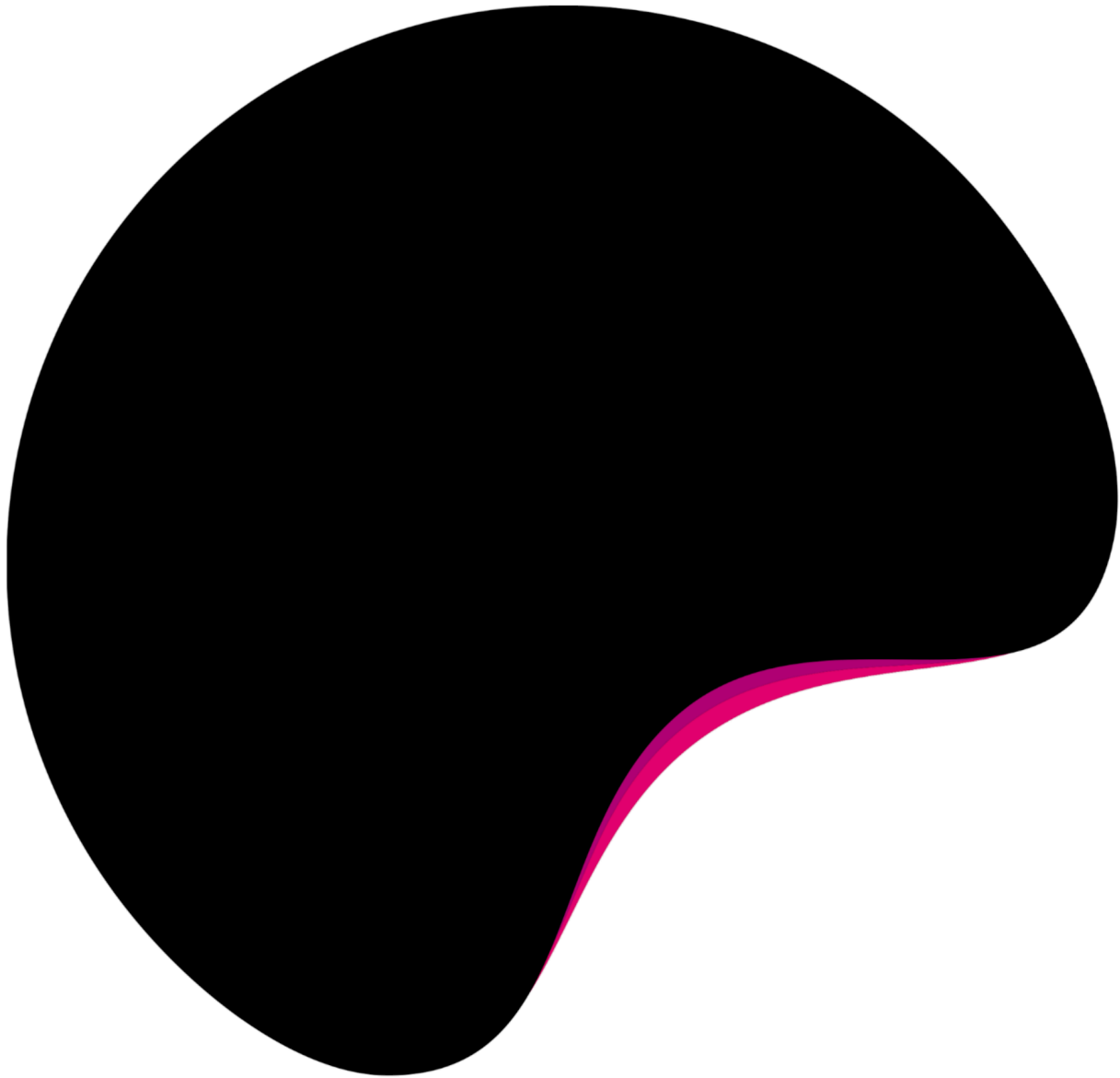




GlowCore

SE Project Documentation

Version: 1.00

Date: 12.04.2026

Git Version: 0e0fa1b

Project Team: Cedric Cathomas
Yoris Kucera
Dominik Wyss
Nathanael Fässler

Project Advisor: Thomas Bocek

School of Computer Science
OST Eastern Switzerland University of Applied Sciences

Contents

I	Management Summary	1
II	Product Documentation	3
1	Glossary	4
2	Requirements	5
2.1	Functional Requirements	5
2.1.1	Actors / Personas	5
2.1.1.1	Casual Player	5
2.1.1.2	Dedicated Player	5
2.1.2	Use Case Diagram	5
2.1.3	Use Cases	7
2.1.4	UC Overview	7
2.2	Non-Functional Requirements	10
2.2.1	NFR Overview	11
2.2.2	Priority Matrix	12
2.2.3	Performance Efficiency	12
2.2.4	Usability	14
2.2.5	Reliability	15
2.2.6	Maintainability	16
2.2.7	Portability	17
3	Domain Analysis	19
3.1	Enemy State Machine	19
4	Architecture	21
4.1	C4 Model	21
4.1.1	System Context Diagram	21
4.1.2	Container Diagram	22
4.1.3	Component Diagram	22
4.1.3.1	Gameplay Logic	23
4.1.3.2	Presentation Layer	24
4.2	Interfaces	25
4.2.1	Overview Diagram	25
4.2.2	Interface Contracts	26
4.2.3	Extensibility through Stable Interfaces	26
4.3	Technologies	27
4.3.1	Unity	27
4.3.2	Universal Render Pipeline (URP)	27
4.3.3	.NET & C#	27
4.3.4	GitHub	27
4.3.5	Blender	27
4.3.6	Audacity	27
4.4	Used Design Patterns	28
4.4.1	Component-Based Architecture	28
4.4.2	Singleton	28
4.4.3	Dependency Injection	28
4.4.4	Scriptable Objects	28
4.4.5	Prefabs	28
4.5	Extensibility	29
5	Quality Measures	30
5.1	Working Environment	30
5.2	Testing	30
5.3	Test Concept	30
5.3.1	Automated Testing (Unit & Integration)	30
5.3.2	System Testing	30

5.3.3	Usability Testing	30
5.3.4	Non-Functional Requirements (NFR) Verification	31
6	Savefile	32

III Project Documentation 33

7	Initial Project Proposal	34
7.1	Team Members	34
7.2	Availabilities	34
7.3	Vision	34
7.4	Proposed Realisation	35
8	Project Plan	36
8.1	Resources	36
8.1.1	People	36
8.1.1.1	Project Management Roles	36
8.1.1.2	Developer Roles	37
8.1.2	Time	37
8.1.3	Cost	37
8.2	Organization	37
8.2.1	Meetings	37
8.2.2	Planning Tools	38
8.3	Schedule	39
8.3.1	Work Items	39
8.3.2	Long Term Plan	39
8.3.2.1	Milestones	39
8.3.2.1.1	Initial Milestones	40
8.3.2.1.2	Updated Milestones as of 07.04.2026	40
8.3.2.2	Epics	41
8.3.2.3	Initial Backlog & MVP	42
8.3.3	Short Term Plan	43
8.3.3.1	Sprint 1 (Sprint Zero) 20.02.2026 - 06.03.2026	44
8.3.3.2	Sprint 2 06.03.2026 - 20.03.2026	45
8.3.3.3	Sprint 3 20.03.2026 - 03.04.2026	46
8.4	Risk Management	46
8.4.1	Risk Matrix	47
8.4.2	Risk Matrix (Mitigated)	47
8.4.3	Assessed Risk Matrix (14.04.2026)	48
8.4.4	Adjustment of Current Risk Assessment	48
9	Time Tracking Report	49
9.1	Time Tracking	49
9.2	Time Statistics	49
10	Personal Reports	50
11	Meeting Minutes	51
11.1	Weekly Scrum / 28.02.2026	51
11.1.1	Agenda	51
11.1.2	Todos	52
11.2	Weekly Scrum and Sprint Retrospective / 06.03.2026	53
11.2.1	Agenda	53
11.2.2	Sprint 1 Retrospective	53
11.2.2.1	What Went Well	53
11.2.2.2	What Didn't Go Well	54
11.2.2.3	Improvements	54
11.2.3	Todos	54
11.3	Sprint Planning / 06.03.2026	55
11.3.1	Sprint goal	55
11.4	Sprint Review (Advisor) / 10.03.2026	56
11.4.1	Agenda	56

11.4.2	Todos	56
11.4.3	Next dates	56
11.5	Weekly Scrum / 13.03.2026	57
11.5.1	Agenda	57
11.5.2	Todos	58
11.6	Weekly Scrum and Sprint Retrospective / 20.03.2026	59
11.6.1	Agenda	59
11.6.2	What went well	59
11.6.3	What didn't	59
11.6.4	Improvements	59
11.6.5	Todos	59
11.7	Sprint Planning / 20.03.2026	61
11.7.1	Sprint goal	61
11.8	Sprint Review (Advisor) / 24.03.2026	62
11.8.1	Agenda	62
11.8.2	Todos	62
11.8.3	Next dates	62
11.9	Weekly Scrum / 27.03.2026	63
11.9.1	Agenda	63
11.9.2	Todos	63
11.10	Weekly Scrum and Sprint Retrospective / 03.04.2026	64
11.10.1	Agenda	64
11.10.2	What went well	64
11.10.3	What didn't	64
11.10.4	Todos	64
11.11	Sprint Planning / 03.04.2026	66
11.11.1	Sprint goal	66
IV	Bibliography	68
V	List of Figures	69
VI	List of Tables	70
VII	Code Listings	72

Part I

Management Summary

Management Summary

Part II

Product Documentation

1 Glossary

Term	Definition
Grid	Refers to the 2D Grid that divides the GlowCore world into rows and columns.
Tile	A Tile is a single unit of space in the grid and has the form of a square.
Node	A Node refers to an object that is occupying a Tile. Not every Tile has a Node, but every Node belongs to a Tile.
World Item	World Items are items that can be picked up by the player and physically exist somewhere in the GlowCore world. In contrast to Nodes they do not belong to a Tile and do not have a hitbox.
Inventory Item	Inventory Items refer to the items that the player possesses. Certain Nodes such as Chests can also hold Inventory Items.
Tool	A Tool is an Inventory Item that can be used to break certain Nodes. For example an Axe can break Trees.
Block	A Block is an Inventory Item that can be placed onto the Grid to build a Node.

Table 1: Glossary

2 Requirements

2.1 Functional Requirements

2.1.1 Actors / Personas

Since a game typically only has one primary actor - **The Player** - it makes sense to define Personas that reflect different types of players. The categorization into Casual- and Dedicated Players covers most player types. In the following, the pronoun **he** is used for simplicity, but the Personas are meant to include all players.

2.1.1.1 Casual Player

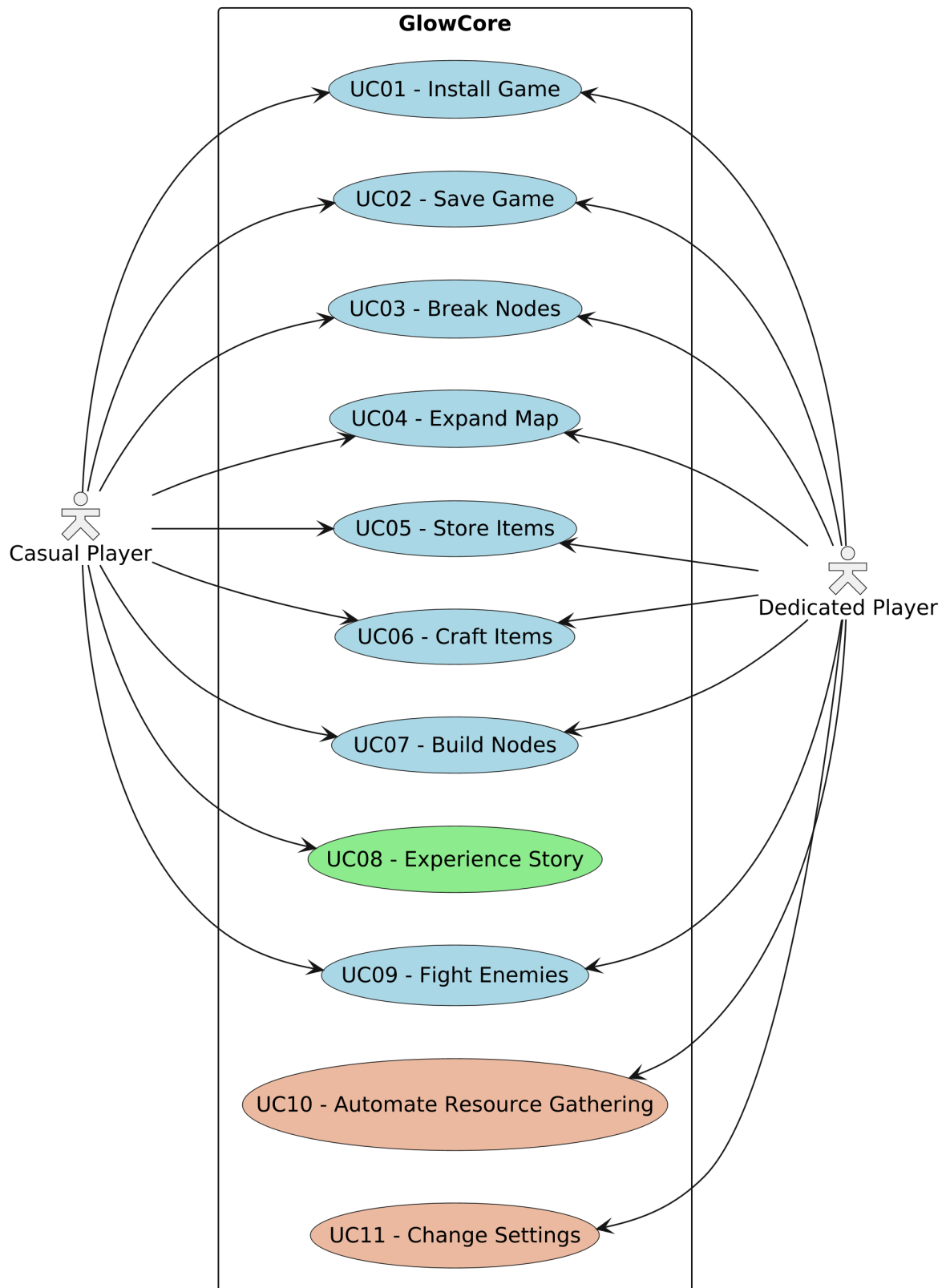
- The Casual Player plays GlowCore infrequently and for short amounts of time. He wants to relax and unwind from his life. He prefers to play the game just for fun without putting effort into it.
- The Casual Player wants to experience the game from beginning to end with a story and explore the world. When he plays the game for the first time, he depends on the game to explain the rules of the game to him without being too overwhelming and tedious. As soon as he finishes the game, he will be done with it, having experienced everything he wanted.
- The Casual Player never changes settings in games, he never feels the need to customize anything. He likes to play the game as it is.

2.1.1.2 Dedicated Player

- The Dedicated Player plays GlowCore in long sessions. He loves games where you build factories and automate everything. He searches for the feeling of progression and finds satisfaction in it. He always searches for a better way to progress and optimize his gameplay.
- The Dedicated Player has already played the game multiple times before, thus he doesn't want to be bored with a long tutorial everytime he starts a new game. He wants to be able to dictate the pacing of the game.
- He doesn't care for the story and lore in games. He doesn't want to read through dialogues and doesn't like long cutscenes.

2.1.2 Use Case Diagram

The diagram shows a modified version of the Use Case Diagram. The Personas take the place of the Actors. An arrow indicates that a Persona uses a feature. A missing arrow means that the Persona is not interested in using a feature, although it remains accessible to him.

**Image 1:** Use Case Diagram

2.1.3 Use Cases

The Use Cases are described in the brief format, with each Use Case including an appropriately detailed description.

It is essential for the project that all Use Cases that define the **MVP** are implemented. The Use Cases marked with **Priority: Middle** are planned to be implemented, but can be reduced if the MVP takes more time than anticipated. The Use Cases marked with **Priority: Low** will be implemented if enough time is left.

Each Use Case table includes a colored status indicator in the top-right corner representing its current verification state:

Color	Status
	Not yet verified (default)
	Implemented – use case fully functional and verified
	Partially implemented – some aspects functional, others pending
	Not implemented – use case not yet functional

Table 2: UC Status Color Legend

2.1.4 UC Overview

The following table provides a compact overview of all Use Cases, their persona, priority, and current verification status.

ID	Name	Persona	Priority	Verify at	Status
UC01	Install Game	Casual & Dedicated Player	High – MVP	M09 Beta	
UC02	Save Game	Casual & Dedicated Player	High – MVP	M09 Beta	
UC03	Break Nodes	Casual & Dedicated Player	High – MVP	M08 Alpha	
UC04	Expand Map	Casual & Dedicated Player	High – MVP	M08 Alpha	
UC05	Store Items	Casual & Dedicated Player	High – MVP	M08 Alpha	
UC06	Craft Items	Casual & Dedicated Player	High – MVP	M08 Alpha	
UC07	Build Nodes	Casual & Dedicated Player	High – MVP	M08 Alpha	
UC08	Experience Story	Casual Player	Low	M10 Release	
UC09	Fight Enemies	Casual & Dedicated Player	Low	M10 Release	
UC10	Automate Resource Gathering	Dedicated Player	Middle	M10 Release	
UC11	Change Settings	Dedicated Player	Middle	M09 Beta	

Table 3: UC Overview – All Use Cases at a Glance

ID	UC01
Name	Install Game
Persona	Casual Player & Dedicated Player
Priority	High - Required for MVP
Description	The player can install the game. The installation requires no technical skills.
Result	

Table 4: UC01 - Install Game

ID	UC02
Name	Save Game
Persona	Casual Player & Dedicated Player
Priority	High - Required for MVP
Description	When the player stops playing, the game saves his save state in a file and loads it when he resumes the game.
Result	

Table 5: UC02 - Save Game

ID	UC03
Name	Break Nodes
Persona	Casual Player & Dedicated Player
Priority	High - Required for MVP
Description	The player can break Nodes in the world with his hands or with tools. The usage of tools e.g. an axe to break a tree, increases the speed significantly. When the Nodes break, they drop items depending on the type of Node. E.g. a Tree drops a Wood-Item. Certain Nodes that are built by the player themselves such as a workbench drop themselves upon breaking.
Result	

Table 6: UC03 - Break Nodes

ID	UC04
Name	Expand Map
Persona	Casual Player & Dedicated Player
Priority	High - Required for MVP
Description	The player is confined to only move on tiles that are unlocked at a given point. The player can drop off items at the GlowCore, whereupon the map expands. The player can then move into and explore the new area.
Result	

Table 7: UC04 - Expand Map

ID	UC05
Name	Store Items
Persona	Casual Player & Dedicated Player
Priority	High - Required for MVP
Description	Items that the player picks up are stored in his inventory. The player's inventory has a limited size. To organize items, the player can move items into chests and take them out again.
Result	

Table 8: UC05 - Store Items

ID	UC06
Name	Craft Items
Persona	Casual Player & Dedicated Player
Priority	High - Required for MVP
Description	The player can craft items into other items with predefined recipes. Without a workbench the player is limited to a few simple recipes. The player can interact with a workstation such as a workbench to unlock more crafting recipes. Workstations such as an oven require additional items: e.g. Coal to smelt an Ingot.
Result	

Table 9: UC06 - Craft Items

ID	UC07
Name	Build Nodes
Persona	Casual Player & Dedicated Player
Priority	High - Required for MVP
Description	The player can place certain items onto the world, where they become Nodes. Each Tile can hold only one Node, and the player is not allowed to build a Node on an already occupied Tile.
Result	

Table 10: UC07 - Build Nodes

ID	UC08
Name	Experience Story
Persona	Casual Player
Priority	Low
Description	The player can experience a story while playing the game. When the game starts there is an intro sequence. There is an end to the game, at which point a cutscene is played. The game contains small narrative hints that fit into the story conveyed by the GlowCore-World, including the acoustic, the visual art style and the overall gameplay experience.
Result	

Table 11: UC08 - Experience Story

ID	UC09
Name	Fight Enemies
Persona	Casual Player & Dedicated Player
Priority	Low
Description	The player can engage in combat with enemies. Defeated enemies drop items that are useful for the player. When the enemy hits the player, he loses hitpoints. When the player has no hitpoints left, he loses some of his items and respawns near the center of the map.
Result	

Table 12: UC09 - Fight Enemies

ID	UC10	
Name	Automate Resource Gathering	
Persona	Dedicated Player	
Priority	Middle	
Description	The player can automate tedious tasks, such as breaking Nodes. He can craft machines that are placed as Nodes and they automatically perform a task e.g. chopping down trees. There is a selection of machines that can be combined to automate more and more. Example: The player places multiple Replanter-Machines in a row which automatically replant trees on the neighbouring tile. The player places an Axe-Machine onto the start of the line, at which point the Axe-Machine chops one tree after another and then moves onto the next tile. To automate it even further, the player places Rotaters on both which cause the Axe-Machine to turn around on contact.	
Result		

Table 13: UC10 - Automate Resource Gathering

ID	UC11	
Name	Change Settings	
Persona	Dedicated Player	
Priority	Middle	
Description	The player can change settings of the game, such as the FOV and the Volume.	
Result		

Table 14: UC11 - Change Settings

2.2 Non-Functional Requirements

This chapter documents the Non-Functional Requirements (NFRs) for the GlowCore project, categorized according to the ISO/IEC 25010:2011 quality model. The ISO/IEC 25010 standard defines a comprehensive set of software quality characteristics that serve as a framework for specifying and evaluating system quality. The following categories are used to structure our NFRs:

- **Performance Efficiency** – NFR1XX
- **Usability** – NFR2XX
- **Reliability** – NFR3XX
- **Maintainability** – NFR4XX
- **Portability** – NFR5XX

Each NFR is documented with a description, concrete acceptance criteria, a measurement method, a verification approach, and a priority. The priority consists of two parts:

- **MVP Relevance:**
 - **Required** – must be fulfilled for the MVP.
 - **Optional** – planned for a later stage, not required for MVP delivery.
- **Importance:**
 - **High** – critical for the product.
 - **Medium** – important but not blocking.
 - **Low** – nice to have.

Each NFR table includes a colored status indicator in the top-right corner representing its current verification state:

Color	Status
	Not yet tested (default)
	Passed – all acceptance criteria met
	Partially met – some criteria passed, others pending
	Failed – acceptance criteria not met

Table 15: NFR Status Color Legend

2.2.1 NFR Overview

The following table provides a compact overview of all non-functional requirements, their category, priority, verification milestone, and current status. This serves as a quick reference before diving into the detailed NFR descriptions below.

ID	Name	Category	Priority	Verify at	Status
NFR101	World Load Time	Performance	Required / High	M08 Alpha	
NFR102	In-Game Transition Time	Performance	Required / High	M08 Alpha	
NFR103	Frame Rate	Performance	Required / High	M08 Alpha	
NFR104	Save File Size	Performance	Required / Medium	M08 Alpha	
NFR105	Minimal Hardware Req.	Performance	Optional / Medium	M09 Beta	
NFR201	Player Restriction Visibility	Usability	Required / High	M08 Alpha	
NFR202	Light Upgrade Progress	Usability	Required / High	M08 Alpha	
NFR203	New Player Learnability	Usability	Required / High	M09 Beta	
NFR204	Input Support	Usability	Optional / Medium	M09 Beta	
NFR205	Game Language	Usability	Optional / Low	M10 Release	
NFR206	Visual Style Consistency	Usability	Optional / Medium	M09 Beta	
NFR301	Save File Portability	Reliability	Required / High	M08 Alpha	
NFR302	Game Progress Persistence	Reliability	Required / High	M08 Alpha	
NFR303	Save Data Integrity	Reliability	Required / High	M08 Alpha	
NFR401	Extensibility of Game Systems	Maintainability	Optional / Medium	M09 Beta	
NFR402	Automated Test Coverage	Maintainability	Required / Medium	M09 Beta	
NFR501	Multi-Platform Support	Portability	Required / Medium	M09 Beta	
NFR502	Engine and Rendering, Pipeline	Portability	Optional / Low	M10 Release	
NFR503	Display Resolution Support	Portability	Optional / Medium	M10 Release	

Table 16: NFR Overview – All Non-Functional Requirements at a Glance

Verification is tied to project milestones (see [UpdatedMilestones](#)) rather than fixed dates to ensure NFRs are checked at meaningful delivery checkpoints:

- **M08 – Alpha Release (10.04.2026)**: All Required / High NFRs must pass. This is the MVP gate.
- **M09 – Beta Release (15.05.2026)**: Required / Medium and Optional / Medium NFRs are verified. Performance tuning and playtesting rounds.
- **M10 – Official Release (05.06.2026)**: Remaining Optional / Low NFRs are verified. Final polish and compliance check.

2.2.2 Priority Matrix

The priority matrix visualizes the distribution of NFRs across the two priority dimensions. This helps identify where the project's quality focus lies and whether the prioritization is balanced.

	Required (MVP)	Optional (Post-MVP)
High	NFR101, NFR102, NFR103 NFR201, NFR202, NFR203 NFR301, NFR302, NFR303	–
Medium	NFR104, NFR402, NFR501	NFR105, NFR204, NFR206 NFR401, NFR503
Low	–	NFR205, NFR502

Table 17: NFR Priority Matrix – MVP Relevance vs. Importance

The matrix shows that the core quality effort is concentrated in the **Required / High** cell (9 NFRs), covering performance, usability, and reliability which are the three pillars that directly impact the player experience in the MVP. The **Optional / Low** cell contains only 2 NFRs that represent project constraints rather than testable quality goals. Priority is on delivering a stable, playable MVP before addressing optional requirements.

2.2.3 Performance Efficiency

Performance Efficiency addresses the amount of resources used under stated conditions. For GlowCore, this primarily concerns load times, frame rates, and resource consumption to ensure a smooth and responsive gameplay experience.

ID	NFR101
Description	World Load Time
Acceptance Criteria	Existing and newly created worlds must load within 3 seconds when the player presses "Start".
Priority	Required / High
Measurement	Measure elapsed time from pressing "Start" until the world is fully loaded and interactive. Tested on minimum hardware specification.
Verification	Automated load-time benchmark test across 10 consecutive loads with varying world states (new and existing). All runs must complete within 3 seconds.
Result	

Table 18: NFR101 – World Load Time

ID	NFR102
Description	In-Game Transition Time
Acceptance Criteria	In-game transitions (e.g. walking into a cave) shall not exceed 2 seconds.
Priority	Required / High
Measurement	Measure elapsed time from triggering the transition until the new scene is fully loaded and interactive.
Verification	Trigger each scene transition during playtesting and record the load time. Automated scene-transition benchmark covering all transition points.
Result	

Table 19: NFR102 – In-Game Transition Time

ID	NFR103
Description	Frame Rate
Acceptance Criteria	The game must run at a minimum of 60 FPS with 1% lows not dropping below 50 FPS on recommended hardware.
Priority	Required / High
Measurement	Use Unity Profiler or an external frame-time analysis tool to record FPS over a 5-minute gameplay session. Evaluate average FPS and 1% low values.
Verification	Run a standardized gameplay scenario on minimum and recommended hardware. Capture FPS metrics and verify thresholds are met.
Result	

Table 20: NFR103 – Frame Rate

ID	NFR104
Description	Save File Size
Acceptance Criteria	Save files should remain compact and use a binary serialization format. Under the currently defined maximum expected game progression and world configuration, a save file should not exceed 5MB.
Priority	Required / Medium
Measurement	Measure the file size of the save file after a full gameplay session under the defined maximum expected game progression and world configuration.
Verification	Create save files at various progression stages and verify the file size remains within the defined limit for the current scope. Confirm that a binary serialization format is used.
Result	

Table 21: NFR104 – Save File Size

ID	NFR105
Description	Minimal Hardware Requirements
Acceptance Criteria	The game must run on minimal hardware, targeting low-end systems to maximize accessibility.
Priority	Optional / Medium
Measurement	Test the game on a defined minimum hardware baseline and verify all performance NFRs (FPS, load times) are met.
Verification	Execute a full gameplay session on minimum-spec hardware. Record FPS, load times, and memory usage to confirm compliance.
Result	

Table 22: NFR105 – Minimal Hardware Requirements

2.2.4 Usability

Usability covers the degree to which the product can be used effectively, efficiently, and satisfactorily. For GlowCore, this includes visual clarity of game mechanics, learnability for new players, and input support.

ID	NFR201
Description	Player Restriction Visibility
Acceptance Criteria	Player restrictions in the game must be clearly communicated through visual indicators such as greyed-out or red-tinted elements. Players must understand restrictions without additional explanation.
Priority	Required / High
Measurement	Conduct playtesting with at least 3 new players. Each player must correctly identify restricted actions without external guidance.
Verification	During playtesting sessions, observe whether players attempt restricted actions and whether they understand the visual cues. Document success rate.
Result	

Table 23: NFR201 – Player Restriction Visibility

ID	NFR202
Description	Light Upgrade Progress Indication
Acceptance Criteria	Light upgrade progress must be clearly indicated to the player at all times during gameplay.
Priority	Required / High
Measurement	During playtesting, ask players to describe their current upgrade progress. At least 80% must answer correctly.
Verification	Conduct playtesting sessions and survey players about their perceived progress. Verify the UI communicates upgrade status unambiguously.
Result	

Table 24: NFR202 – Light Upgrade Progress Indication

ID	NFR203
Description	New Player Learnability
Acceptance Criteria	A new player shall understand core mechanics (movement, gathering, upgrading) within 5 minutes without external instructions.
Priority	Required / High
Measurement	Time how long a new player takes to perform their first gather, move to a new area, and initiate an upgrade without any external help.
Verification	Conduct playtesting with at least 3 players who have never seen the game. Measure time to complete core actions. All players must achieve this within 5 minutes.
Result	

Table 25: NFR203 – New Player Learnability

ID	NFR204
Description	Input Support
Acceptance Criteria	The game must support both controller and keyboard input methods.
Priority	Optional / Medium
Measurement	Test all core gameplay actions with both a standard controller (e.g. Xbox controller) and keyboard. All actions must be executable with both input methods.
Verification	Perform a full gameplay session using only controller, then only keyboard. Verify all interactions, menus, and gameplay mechanics are fully accessible with each input method.
Result	

Table 26: NFR204 – Input Support

ID	NFR205
Description	Game Language
Acceptance Criteria	All in-game text, UI elements, and instructions must be in English.
Priority	Optional / Low
Measurement	Review all in-game text and UI elements for non-English content.
Verification	Manual review of all text-containing assets and UI screens to confirm English language throughout.
Result	

Table 27: NFR205 – Game Language

ID	NFR206
Description	Visual Style Consistency
Acceptance Criteria	All visual assets must follow the low-poly pastel art style consistently throughout the game.
Priority	Optional / Medium
Measurement	Visual review of all assets against the defined art style guide. No asset shall deviate from the low-poly pastel aesthetic.
Verification	Conduct an art review session where all in-game assets are compared against reference material. Flag and correct any deviations.
Result	

Table 28: NFR206 – Visual Style Consistency

2.2.5 Reliability

Reliability addresses the degree to which a system performs specified functions under specified conditions for a specified period of time. For GlowCore, this is critical for save data integrity and game stability.

ID	NFR301
Description	Save File Portability
Acceptance Criteria	Each world save must be stored as a single file that can be copied and moved. When placed into the game's save file folder, the save file must be recognized and function without issues.
Priority	Required / High
Measurement	Copy a world save file to a different machine or location, place it in the save folder, and verify the game loads it correctly with all progress intact.
Verification	Create a world save file with significant progress. Copy it to a new installation. Load the game and verify all progress, items, and world state for that world are preserved.
Result	

Table 29: NFR301 – Save File Portability

ID	NFR302
Description	Game Progress Persistence
Acceptance Criteria	Game progress shall not be lost during normal shutdown. Auto-save shall occur at least every 2 minutes or immediately after a light upgrade.
Priority	Required / High
Measurement	Verify auto-save triggers by monitoring save file timestamps during gameplay. Confirm no data loss after normal shutdown.
Verification	Play for several minutes, perform a light upgrade, then shut down the game normally. Restart and verify all progress is retained. Additionally, verify auto-save timestamps occur within 2-minute intervals.
Result	

Table 30: NFR302 – Game Progress Persistence

ID	NFR303
Description	Save Data Integrity
Acceptance Criteria	Save data must persist correctly across sessions without corruption.
Priority	Required / High
Measurement	Perform repeated save/load cycles (minimum 20) and verify data integrity after each cycle by comparing expected vs. actual game state.
Verification	Automated test that saves game state, reloads it, and compares all serialized fields for correctness. Run across multiple sessions and verify zero corruption occurrences.
Result	

Table 31: NFR303 – Save Data Integrity

2.2.6 Maintainability

Maintainability represents the degree of effectiveness and efficiency with which a product can be modified. For GlowCore, this covers extensibility of game systems and test coverage.

ID	NFR401
Description	Extensibility of Game Systems
Acceptance Criteria	The code must allow adding new LightCore upgrades and new items with their resource nodes without requiring code restructuring.
Priority	Optional / Medium
Measurement	A developer unfamiliar with the codebase must be able to add a new upgrade or item by following existing patterns, without modifying core system code.
Verification	Task a team member with adding a new test upgrade and resource node. Measure the time required and verify no structural code changes were necessary.
Result	

Table 32: NFR401 – Extensibility of Game Systems

ID	NFR402
Description	Automated Test Coverage
Acceptance Criteria	All systems must be covered by automated tests. The project targets a minimum of 40% code coverage. (This coverage target is relatively common for video games)
Priority	Required / Medium
Measurement	Measure code coverage using the Unity Code Coverage package. Report overall and per-system coverage percentages.
Verification	Run the full automated test suite and generate a coverage report. Verify that overall coverage meets or exceeds 40%.
Result	

Table 33: NFR402 – Automated Test Coverage

2.2.7 Portability

Portability addresses the degree of effectiveness and efficiency with which a system can be transferred from one environment to another. For GlowCore, this covers platform support and display resolution compatibility.

ID	NFR501
Description	Multi-Platform Support
Acceptance Criteria	The game must run on Windows, Linux, and Web platforms.
Priority	Required / Medium
Measurement	Build and deploy the game on all three platforms. Launch and complete a full gameplay session on each.
Verification	Execute the full test suite and a manual gameplay session on Windows, Linux, and a Web browser. Verify all features work identically across platforms.
Result	

Table 34: NFR501 – Multi-Platform Support

ID	NFR502
Description	Engine and Rendering Pipeline
Acceptance Criteria	The game must run on the Unity engine using the PhysX physics engine and the Universal Render Pipeline (URP).
Priority	Optional / Low
Measurement	Verify project settings confirm Unity with PhysX and URP are configured as the active engine and rendering pipeline.
Verification	Review Unity project settings and confirm PhysX is the active physics engine and URP is the active render pipeline. Verify no fallback to built-in pipeline occurs during gameplay.
Result	

Table 35: NFR502 – Engine and Rendering Pipeline

ID	NFR503
Description	Display Resolution Support
Acceptance Criteria	The game must support common display resolutions with a 16:9 aspect ratio. Minimum: 1280×720. Recommended: 1920×1080, 2560×1440, and 3840×2160 (4K).
Priority	Optional / Medium
Measurement	Launch the game at each supported resolution and verify the UI scales correctly, no elements are clipped, and the game renders properly.
Verification	Test the game at all specified resolutions. Take screenshots and verify correct rendering, UI scaling, and absence of visual artifacts.
Result	

Table 36: NFR503 – Display Resolution Support

3 Domain Analysis

Below is the domain model diagram for the problem domain of our project. Due to the nature of this project, the domain is modeled using an entity-relationship diagram, which captures the key entities and their relationships within the system. The diagram provides a high-level overview of the main components and their interactions, which is essential for understanding the structure of the problem domain.

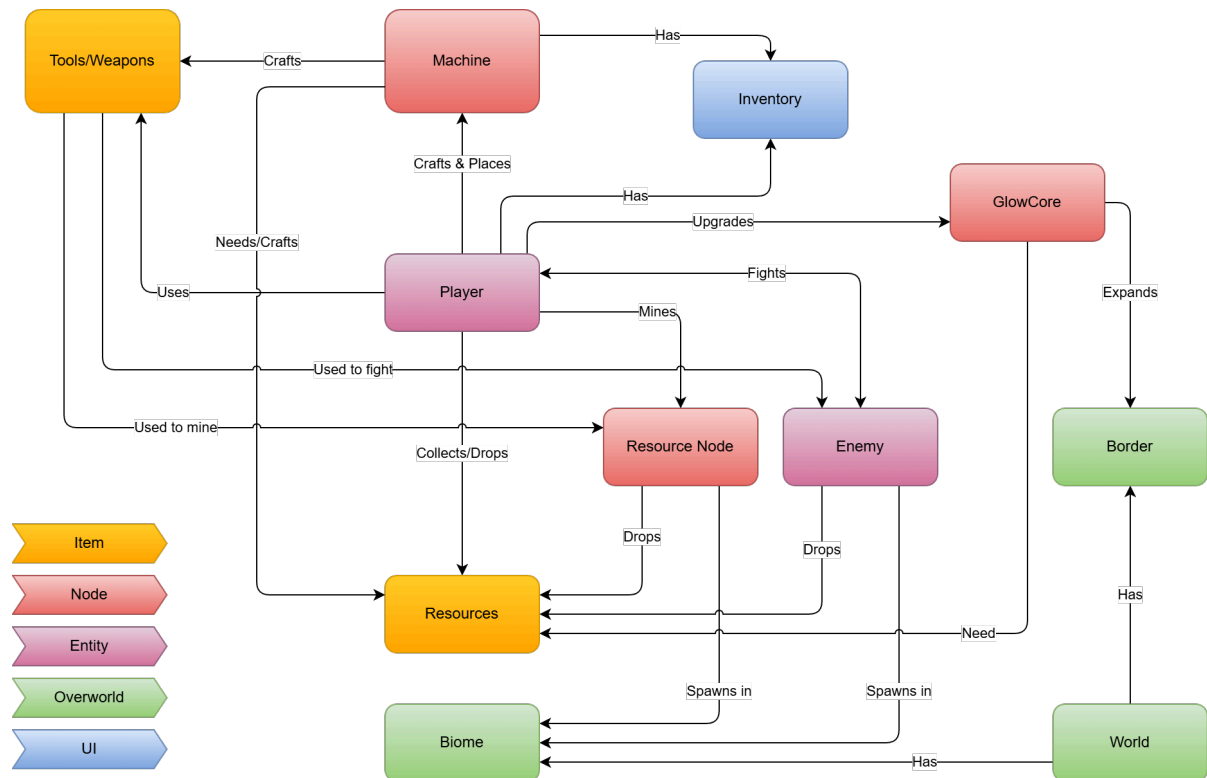


Image 2: Domain Model Diagram

One of the key aspects of the domain is the relationship between “Nodes” and “Items”. Some items can be placed in the world which turns them into Nodes. This means that the same concept can have different representations and behaviors depending on the context.

3.1 Enemy State Machine

The below diagram represents the state machine for the enemy behavior in our project. It captures the various states an enemy can be in as well as the transitions between those states and the conditions that trigger those transitions.

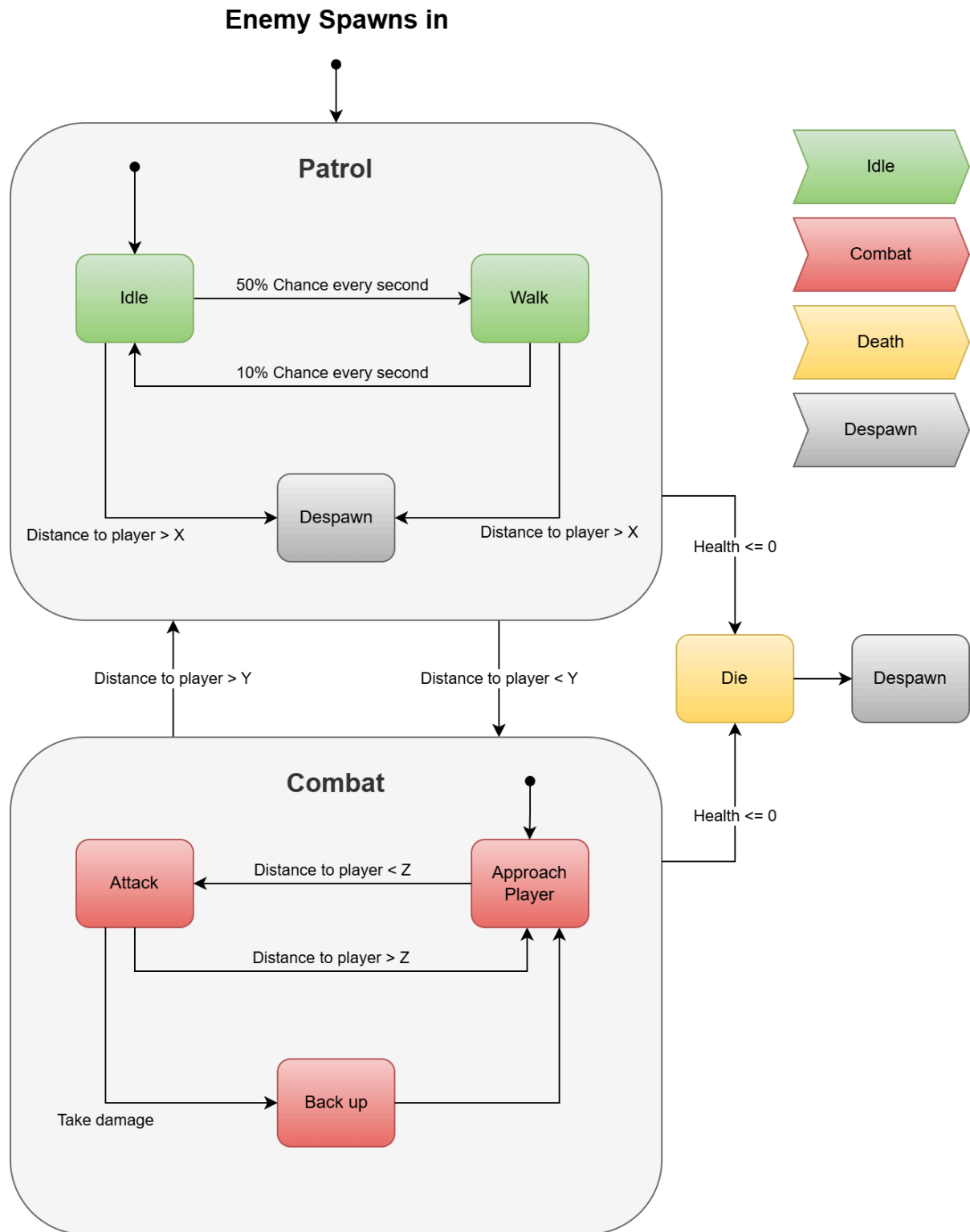


Image 3: Enemy State Machine Diagram

4 Architecture

4.1 C4 Model

To model the architecture of GlowCore we decided to use the C4 Model. The model is designed to provide stakeholders new to the project a high-level overview of GlowCore with different levels of abstraction. We decided against arc42 because it is too extensive for this project.

We intentionally omitted the Code Diagram (Layer 4 of C4) in this documentation because of how much complexity it would add without providing much value. This is common in project management to omit the Code Diagram.

4.1.1 System Context Diagram

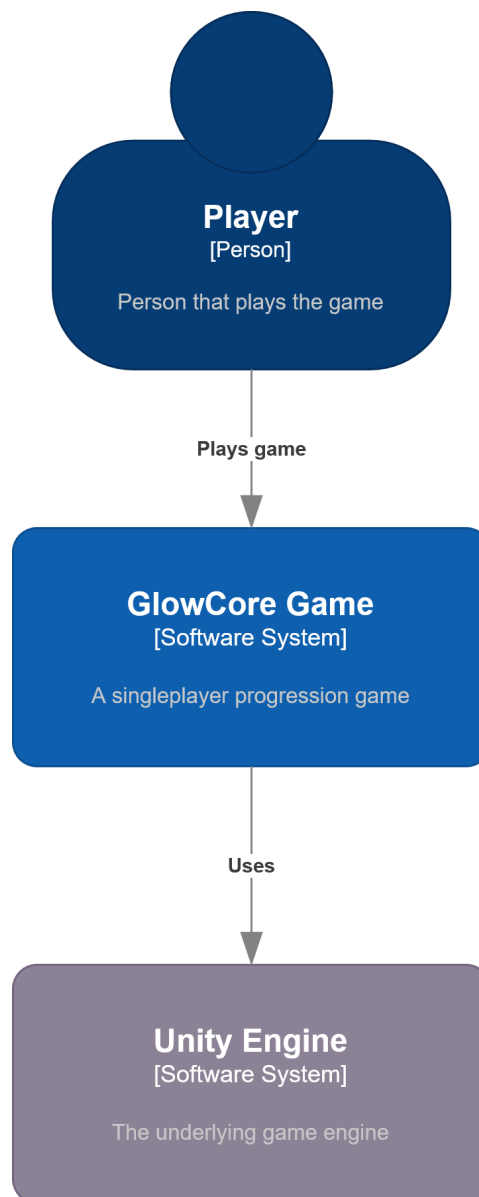


Image 4: C4 System Context Diagram

4.1.2 Container Diagram

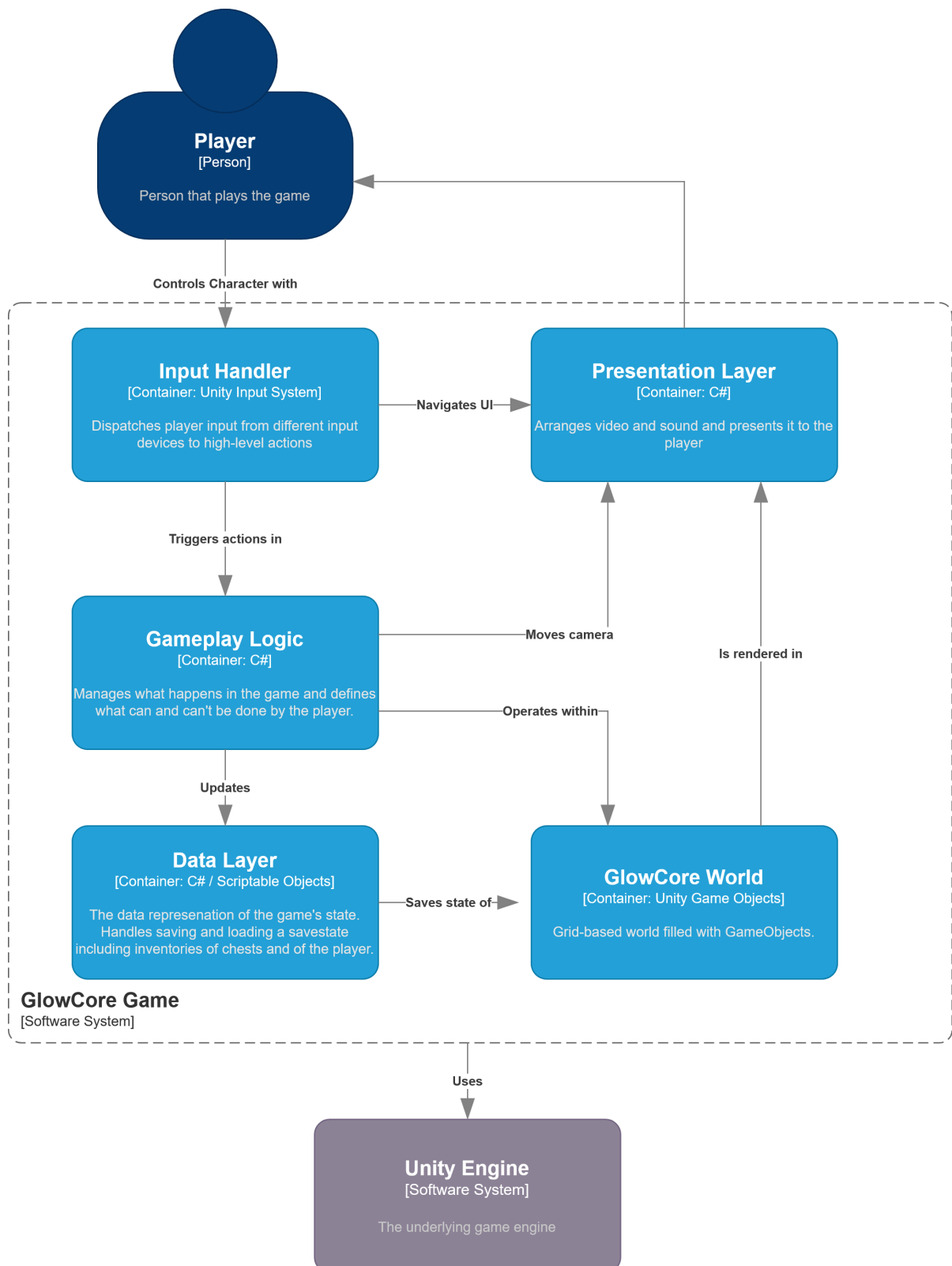


Image 5: C4 Container Diagram

4.1.3 Component Diagram

The Component Diagram zooms into selected Containers to provide a deeper overview.

4.1.3.1 Gameplay Logic

A naive approach to implement features like Breaking Nodes, Building, Inventories and Crafting would be to just implement them separately, each with its own isolated input handling and execution logic. This would result in duplicated code and poorly maintainable code.

Instead, we analyzed the features and abstracted them into two groups: **Use Actions** and **Interact Actions**.

Use Actions are responsible for using the Item in the players hand (The item that is selected in the hotbar and visually displayed in the players hand). The player can only break Nodes when he holds a tool in his hand and he can only place Nodes if he holds a Block in his hand.

Interact Actions define the behaviour of hovered over Nodes. For example the player opens a chest by hovering over it with the cursor and pressing the corresponding button. The GlowCore and all crafting stations share the same input to avoid code duplication. This approach also makes our game extensible.

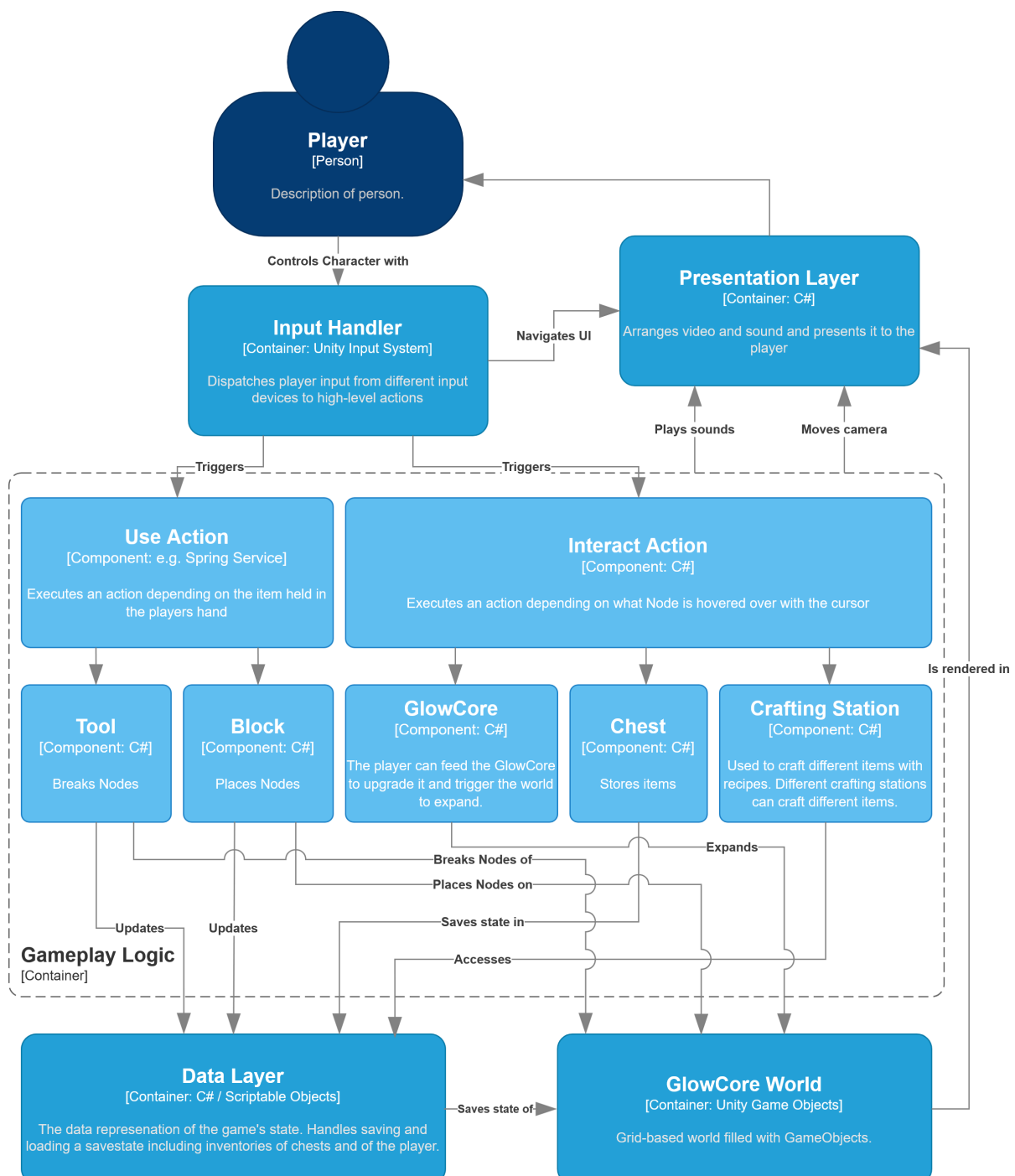


Image 6: C4 Component Diagram - Gameplay Logic

4.1.3.2 Presentation Layer

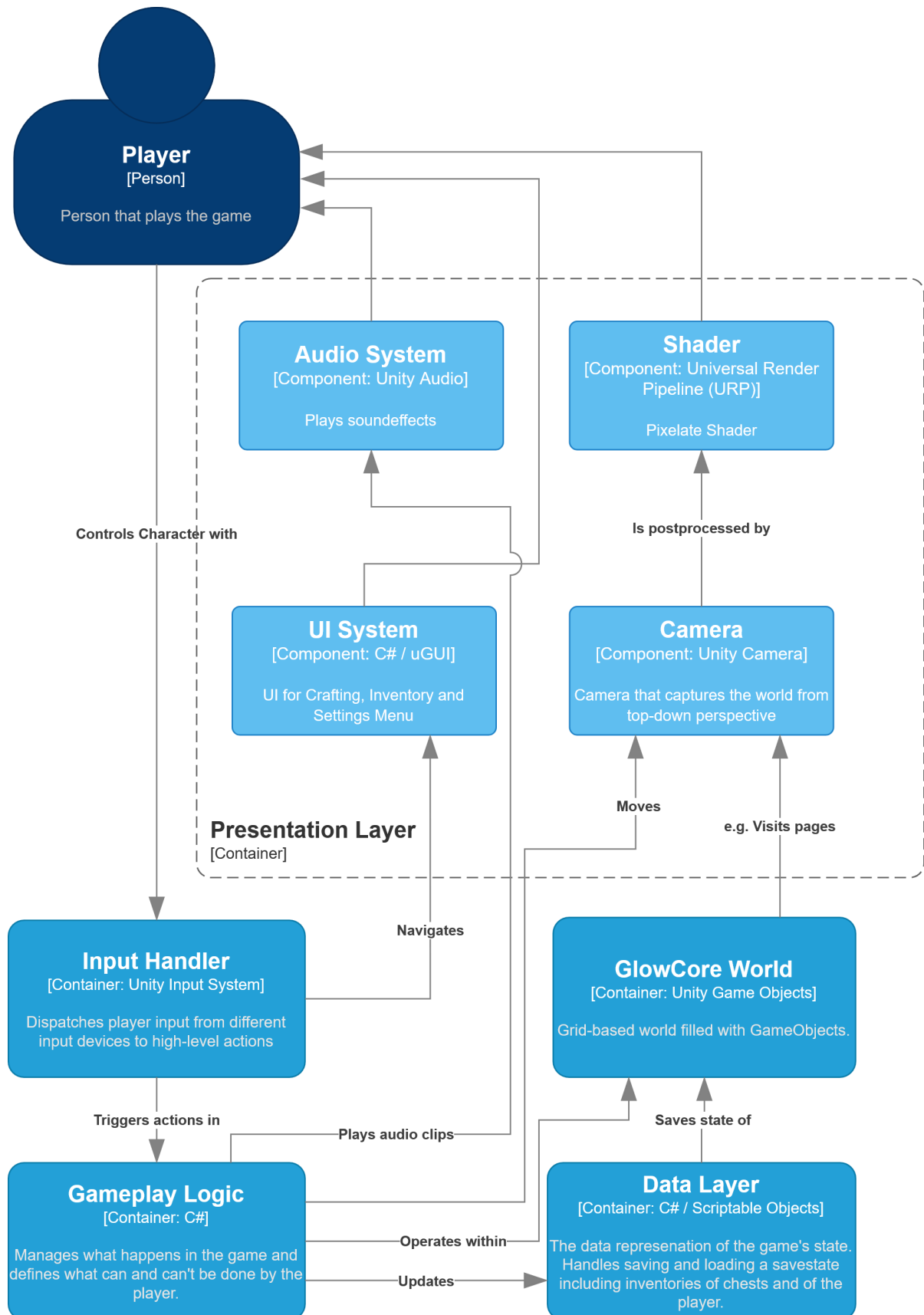


Image 7: C4 Component Diagram - Presentation Layer

4.2 Interfaces

GlowCore's container boundaries are expressed through either a C# interface or a ScriptableObject asset reference, with the exception of the input boundary where Unity's own message dispatch handles the connection directly.

4.2.1 Overview Diagram

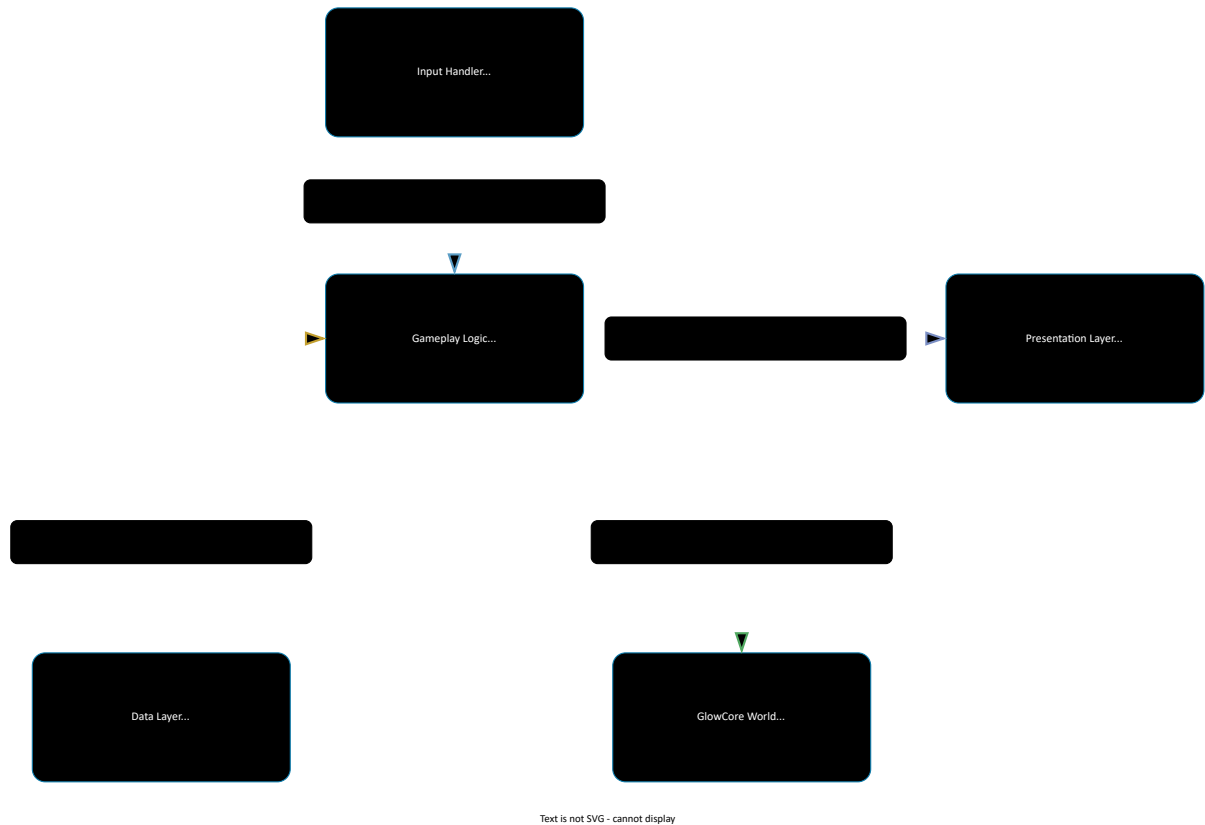


Image 8: Interface Overview Diagram

4.2.2 Interface Contracts

Interface <i>Boundary Mechanism</i>	Purpose
Container Boundaries	
Unity Input callbacks <i>Input Handler → Gameplay Logic</i> Unity Input System (no C# interface)	Unity calls methods like <code>OnUse</code> , <code>OnInteract</code> and <code>OnMove</code> directly on <code>MonoBehaviour</code> components such as <code>PlayerHand</code> and <code>NodeActionSystem</code> . There is no C# interface at this boundary. Unity's own message dispatch handles the connection.
IInteractable <i>Gameplay Logic → GlowCore World</i> C# interface	Nodes that do something when the player interacts with them implement this interface. A Chest opens its inventory, a <code>CraftingTableInteractable</code> shows the crafting UI, and <code>GlowCoreObject</code> runs its own logic, all through the same single-method contract.
IInventoryService <i>Gameplay Logic → Presentation Layer</i> C# interface + events (<code>Action<T></code>)	The main contract between the inventory system and the UI. Components like <code>InventoryUI</code> and <code>HotbarUI</code> subscribe to events such as <code>OnSlotChanged</code> and <code>OnHotbarSelectionChanged</code> to stay in sync with the game state, without ever referencing <code>PlayerInventory</code> directly.
ICraftingService <i>Gameplay Logic → Presentation Layer</i> C# interface + event (<code>Action</code>)	Gives the crafting UI everything it needs: the current recipe list, availability checks, the <code>Craft()</code> command to actually execute a recipe, and the <code>OnRecipesRefreshed</code> event that triggers a refresh whenever the player's inventory changes.
ScriptableObject asset refs <i>Data Layer → Gameplay Logic</i> Unity serialised reference (<code>[SerializeField]</code>)	<code>Item</code> , <code>NodeData</code> and <code>Recipe</code> assets are set up in the Unity Inspector and read at runtime by the components that need them. There is no code dependency; the connection is entirely through serialised Unity references.
Gameplay Logic - Internal Component Boundaries	
IHandItem <i>PlayerHand → Item Prefab (GL internal)</i> C# interface	Any item that can be used while held in the player's hand implements this interface. When the player presses the use button, <code>PlayerHand</code> calls <code>Use()</code> on whichever component is currently active, without caring whether it is a <code>ToolBehaviour</code> , a <code>BlockBehaviour</code> , or anything else.
IPlayerInventoryAware <i>PlayerHand → Item Prefab (GL internal)</i> C# interface	Some item prefab components need access to the player's inventory to function correctly. This interface lets <code>PlayerHand</code> pass them a reference when the item is equipped, avoiding any need for a singleton lookup.

Table 37: Interface Contracts

4.2.3 Extensibility through Stable Interfaces

These interfaces are what makes NFR401 achievable in practice. Each interface was designed so that adding something new only requires work in one place.

Adding a new usable item means implementing `IHandItem` on a prefab component. Adding a new interactive node means implementing `IInteractable`. In both cases nothing else in the system needs to change: the Input Handler, the Presentation Layer, and the Data Layer are completely unaware of the new addition.

Extending the item and crafting catalogue is just as simple. A new `Item` or `NodeData` asset can be created as a `ScriptableObject`, filled in through the Unity Inspector, and referenced wherever it is needed, without touching any C# code at all. The same applies to crafting recipes: a new `Recipe` asset is all that is needed to make a new combination available in the crafting UI.

The inventory and crafting UI are equally easy to extend. Because `InventoryUI` and `HotbarUI` depend only on `IInventoryService`, and `CraftingTableUI` only on `ICraftingService`, a new UI component for either system can be added by subscribing to the same events and use given methods, without modifying any existing class.

4.3 Technologies

4.3.1 Unity

The most important decision was the selection of the game engine. We decided to use Unity because we had the most experience in it and all members have already learned C#. Had our game been 2D and not 3D, Godot would have been our preferred choice.

4.3.2 Universal Render Pipeline (URP)

Unity provides 3 rendering pipelines:

- **Built-in Render Pipeline:** Legacy render pipeline
- **Universal Render Pipeline (URP):** Lightweight scriptable render pipeline.
- **High Definition Render Pipeline (HDRP):** For realistic surfaces.

We chose URP because the HDRP would be overkill for our project and the Built-in Render Pipeline is deprecated and new projects should use URP instead. URP is perfectly suitable to create custom shaders and viewports.

4.3.3 .NET & C#

Unity's default scripting language is C#, so this decision came bundled with the selection of the game engine. Although it is possible to use Plugins to run other languages such as e.g. C++, there are no significant benefits, so we settled on C#.

Unity uses Mono, a .NET runtime to create cross-platform applications. We use NUnit via the Unity Testing Framework for automated testing.

4.3.4 GitHub

We use GitHub for version control. At the beginning of the project we used GitLab, but we switched to GitHub because we experienced problems setting up the CI/CD Pipeline to work. The documentation on GitLab was limited, since most tutorials and examples are focused on GitHub Actions.

4.3.5 Blender

To create our 3D Models we use Blender. Blender is free and open-source and offers all features of its competitors, so the choice was obvious. The workflow to import 3D Models from Blender into Unity is great and preserves all important informations.

4.3.6 Audacity

We use Audacity to record and edit our soundeffects. Audacity is free, easy to use and provides all necessary tools for simple sound design, so there was no need to choose a more sophisticated audio editing software.

4.4 Used Design Patterns

4.4.1 Component-Based Architecture

We use Unity's Component-Based Architecture, where every `GameObject` has Components that define its behaviour. To implement custom Components, we use MonoBehaviour-Scripts to define the logic of a `GameObject`. This approach increases modularity because it is possible to split the logic into different Components and reuse them.

We don't have a real **Entity Component System (ECS)** because we don't separate the logic from the data. To support a real ECS we would need to use Unity's **Data-Oriented Technology Stack (DOTS)**. For our project the additional complexity of DOTS would outweigh its performance benefits.

4.4.2 Singleton

We use the Singleton Pattern where appropriate. In our game it definitely makes sense because we have systems that only exist once. For example there is only one Player and only one World-Grid.

4.4.3 Dependency Injection

Games often have tightly coupled systems. To counter this we use dependency injection wherever appropriate to decrease coupling.

4.4.4 Scriptable Objects

In our game we have to define and store data about Items and Nodes in a organized way. Meant are the Item/Node-Definitions, not the instances at runtime. For example for the Wood-Item we need to store its ID, 3D Model, Max Stack-Size etc..

Scriptable Objects solve this problem. Scriptable Objects are objects that are not bound to a `GameObject`. Instances of ScriptableObjects can be saved as **.asset** files. This improves Version Control for Items/Nodes since they are saved as individual files.

4.4.5 Prefabs

We use Prefabs to save specific `GameObjects` as **.prefab** files. This enables us to dynamically Instantiate Prefabs as `GameObjects` during runtime, which is necessary for e.g. Building.

4.5 Extensibility

Our game is designed to be extensible and exchangeable wherever feasible. Thanks to our ScriptableObject Architecture we can easily add new Items/Nodes just by creating a new Instance of the corresponding ScriptableObject and setting the parameters and referencing the 3D Model and Icon.

Thanks to the Component-Based Architecture we can easily add new Items that perform a specific action when used by the players. It would be easy to add for example Consumables. An additional benefit that it is easy to add Nodes with custom behaviour for when the player interacts with it. For example you could just create a Signpost component and add it to a Node to create a Node where you can write something on it and read it.

We save our 3D Models separate from the ScriptableObjects in the .fbx format to improve exchangeability. It is possible to only exchange the 3D Model without touching the ScriptableObject.

A major limit of our architecture that we are strongly tied to Unity. Switching to another game engine would require a significant effort due to how our systems are relying on Unity's systems.

5 Quality Measures

5.1 Working Environment

5.2 Testing

5.3 Test Concept

Our testing strategy for GlowCore is fundamentally shaped by the nature of the project. Since GlowCore is a standalone Unity game built for Windows, Linux, and WebGL, we don't have any server connections, huge amount of data in a database or a frontend beyond simple game-menus. This allows us to focus our entire testing effort directly on the client-side application, ensuring gameplay mechanics, performance, and user experience are as solid as possible.

The test concept is divided into automated testing for our codebase, manual system verification, extensive usability testing, and targeted checks for our non-functional requirements (NFRs).

5.3.1 Automated Testing (Unit & Integration)

At the code level, we rely heavily on the Unity Test Framework (UTF). We use it to write both unit tests and integration tests that verify whether individual functionalities and interconnected systems are working correctly.

A core part of our workflow is our CI/CD pipeline. Our UTF tests are set up to run automatically on the pipeline every time someone pushes new code. Because these tests run continuously on the latest codebase, they effectively act as our regression tests. They give us immediate feedback if a new feature or bug fix accidentally breaks an older, already working part of the game. We are aiming for a minimum of 40% automated test coverage across our systems, which is a pretty standard and realistic target for game development projects.

5.3.2 System Testing

We don't do automated system tests. Fully automating the entire game loop in Unity is just very difficult. It usually results in tests that break way too easily, for example, just because a small visual asset was moved slightly.

Instead, system testing is handled manually. For this, we regularly take the automated WebGL build generated from the current pipeline state and compare it directly against a known, stable release build. This manual side-by-side verification helps us ensure that the game as a whole still compiles, runs, and functions as expected before we consider a new version stable.

5.3.3 Usability Testing

There is a strict limit to what we can test automatically in a game project. A script might tell you if a player's health goes down correctly, but it can't tell you if the game is actually fun or if the controls make sense. Because of this, usability testing is a massive aspect of our QA process.

We will conduct regular playtesting sessions where we let several new users play the game without giving them any prior instructions. Our main goal here is to find out what feels good, what is frustrating, and if the core mechanics are actually understandable. For instance, we want to see if a brand-new player can figure out how to move, gather resources, and upgrade within the first 5 minutes. We will also observe if players naturally understand the visual cues for restricted actions (like red-tinted UI elements) or if they can accurately track their light upgrade progress.

5.3.4 Non-Functional Requirements (NFR) Verification

A huge portion of our testing isn't just about finding bugs, but verifying our NFRs. We have defined a wide spectrum of NFRs covering different quality attributes. They are sorted by priority, meaning that while our ultimate goal is to fulfill all of them, the ones marked as "Required" are strict targets we have to hit for the final submission.

We cover NFR testing through a mix of automated benchmarks and manual playtesting protocols:

- **Performance Efficiency:** We regularly check our performance manually to make sure the game runs smoothly. We aim to ensure that the world loads quickly, in-game scene transitions are fast, and the game maintains a solid 60 FPS on recommended hardware.
- **Reliability:** Since saving the game correctly is critical, we manually test save and load cycles to ensure data doesn't corrupt. We also verify that save files remain small enough (under 5MB), that they are fully portable between machines, and that auto-saves trigger correctly.
- **Portability & Maintainability:** We manually verify that the game runs identically across Windows, Linux, and WebGL, and that it supports standard 16:9 resolutions. From a code perspective, we constantly test extensibility during normal development. We frequently add new elements like resource nodes, items, or recipes based on existing systems. If our internal architecture was too complicated to extend, we would notice it very quickly.
- **Usability & Visuals:** Beyond the playtesting mentioned earlier, we also manually review the game's language to ensure all text is in English. We test controller and keyboard inputs by simply playing with both, and we hold art reviews to make sure all new assets fit our low-poly pastel style guide.

By combining the automated safety net of our UTF pipeline with heavy manual playtesting and strict NFR benchmarking, we believe this concept will keep the game stable while still allowing us to iterate quickly on the gameplay.

6 Savefile

The game periodically saves the player's progress in a savefile. This file contains all the necessary information to restore the game state when the player loads it again. It is stored as a binary file on the player's device. The following diagram illustrates the structure of the savefile:

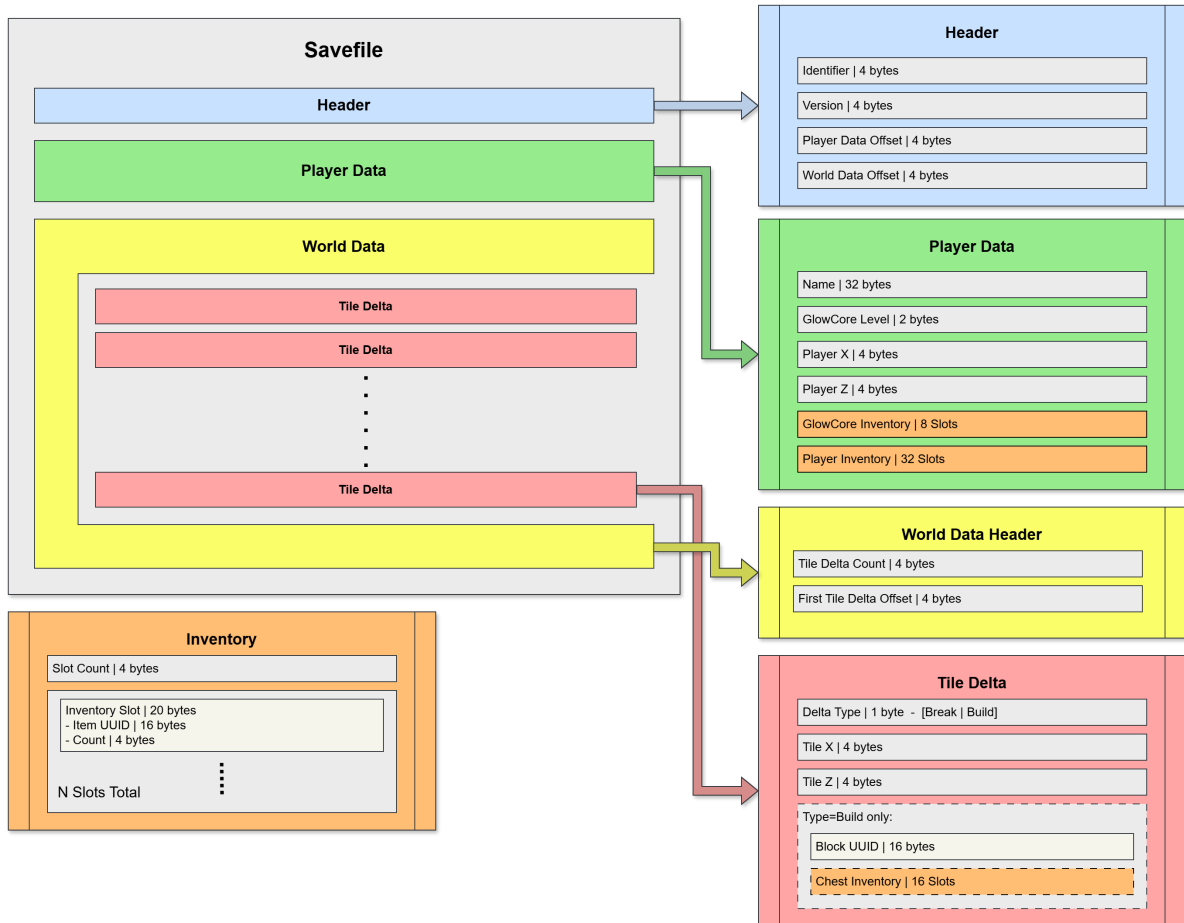


Image 9: Savefile Structure

The savefile is organized into several sections, each containing specific types of data:

- **Header:** Contains metadata about the savefile, such as an identifier, version number, and offsets to subsequent sections.
- **Player Data:** Stores information about the player itself, including name, GlowCore level, inventory, etc.
- **World Data:** Contains information about the game world itself, such as where nodes have been placed or destroyed.

The world data is stored as a list of deltas, instead of a full snapshot of the world state. Each delta represents a change to the world from its initial state, such as placing or destroying a node. This approach allows for more efficient storage and faster loading times, as only the changes need to be recorded rather than the entire world state.

Part III

Project Documentation

7 Initial Project Proposal

Project name: GlowCore

Date: 19.12.2025

7.1 Team Members

1. Cedric Cathomas (Cedric.Cathomas@ost.ch)
2. Dominik Wyss (Dominik.Wyss1@ost.ch)
3. Nathanael Fässler (Nathanael.Faessler@ost.ch)
4. Yoris Kucera (Yoris.Kucera@ost.ch)

7.2 Availabilities

Time slot	Mon	Tue	Wed	Thu	Fri
08h00–09h00	-	(XR)	-	-	(XO)
09h00–10h00	-	(XR)	-	-	(XO)
10h00–11h00	(XR)	-	-	-	(XO)
11h00–12h00	(XR)	-	-	-	(XO)
12h00–13h00	-	-	-	-	(XO)
13h00–14h00	XR	-	-	-	(XO)
14h00–15h00	XR	-	-	-	(XO)
15h00–16h00	(XR)	XR	-	-	(XO)
16h00–17h00	(XR)	XR	-	-	(XO)
17h00–18h00	-	(XR)	-	-	(XO)
18h00–19h00	-	(XR)	-	-	-

Table 38: Team availability for review meetings

7.3 Vision

GlowCore is a singleplayer progression game focused on resource gathering and gradual world expansion. At the start, the player can only access a small, limited area of the game world, while the rest remains hidden and unreachable. At the center of the world is a special entity that can be upgraded using collected materials. Each upgrade expands the accessible area, revealing new parts of the world and enabling further progression. The core gameplay loop is built around exploring newly unlocked areas, gathering resources, and using them to continuously improve the central entity and push the boundaries of the world outward.

As the world expands, additional gameplay systems deepen the experience. The player can obtain and use different materials, interact with tools, craft items, and face enemies within the expanding world. More advanced mechanics such as machines for processing resources, farming systems, and specialized equipment can be introduced over time to increase complexity and strategic depth.

7.4 Proposed Realisation

The project will be implemented using Unity as the primary game engine, using its integrated systems for rendering, physics, input handling, animation, audio, and crossplatform deployment. Development will be done in C# using Unity's component-based architecture (GameObjects, MonoBehaviours, and ScriptableObjects) to structure gameplay logic and systems. Unity's built in physics engine and animation system (Animator) will be used to realise core gameplay and user interaction. Visual assets will be rendered using Unity's Universal Render Pipeline (URP). Individual systems will be tested for robustness using unit tests by method of Unity's own testing framework, based on NUnit. Version control will be handled with Git. The project will be developed within the Unity Editor.

8 Project Plan

8.1 Resources

8.1.1 People

The project team consists of 4 students. Each member contributes approximately 8.6 hours per week and is responsible for tasks of their defined roles.

8.1.1.1 Project Management Roles

Role	Description	Names
Product Owner	• Define product vision • Manage backlog • Prioritize features • Validate requirements	Nathanael Fässler
Scrum Master	• Facilitate meetings • Remove blockers • Ensure Scrum process • Support team workflow	Dominik Wyss
Project Manager	• Schedule planning • Resource management • Risk control • Progress reporting	Cedric Cathomas
DevOps Engineer	• CI/CD maintenance • Build automation • Deployment support • Infrastructure management	Yoris Kucera

Table 39: Project Management Roles

8.1.1.2 Developer Roles

Role	Description	Names
Lead Developer	• Overall technical coordination • Code quality control • Architecture decisions • Integration management	Yoris Kucera
Developer	• Gameplay implementation • Bug fixing • Feature development • System integration • Writing Test Cases	Dominik Wyss Yoris Kucera Nathanael Fässler Cedric Cathomas
Art Director / 3D Artist	• Asset creation • Scene design • Visual consistency • Asset optimization	Nathanael Fässler
UI Architect	• UI/UX design • Menu systems • HUD implementation • User interaction flow	Cedric Cathomas
Graphics Programmer	• Shader development • Particle effects • Performance optimization • Visual effects integration	Yoris Kucera
Audio Engineer	• Sound design • Audio integration • Music management • Audio balancing	Dominik Wyss
Level Designer	• Level layout • Gameplay balancing • Difficulty tuning • Playtesting feedback	Nathanael Fässler Cedric Cathomas

Table 40: Developer Roles

8.1.2 Time

The project runs from 16.02.2026 to 12.06.2026 with an estimated effort of 120 hours per person, resulting in a total workload of 480 hours, structured in milestones and sprints.

8.1.3 Cost

Most required tools and platform are freely available through free or student licenses. We plan on publishing GlowCore on steam for which steam charges a 100 US-Dollar publishing fee. The cost will be shared by the members.

8.2 Organization

8.2.1 Meetings

The project meetings follow mainly the Scrum standard. With weekly Scrum meetings replacing daily stand-ups and biweekly sprint planning including retrospectives. Also the Regular advisor meetings to present and discuss our results so far.

Type	Todos	Frequency
Sprint Review (Advisor)	• Present sprint results • Demonstrate features • Discuss open issues • Collect advisor feedback • Document decisions	Every 2 weeks Tuesday 15:00 - 16:00
Weekly Scrum	• Report progress • Define next steps • Identify blockers • Update tasks • Support team members	Weekly Friday 14:00 - 15:00
Sprint Retrospective	• Analyze sprint process • Identify problems • Define improvements • Document lessons learned	Every 2 weeks Friday 15:00 - 16:00
Sprint Planning	• Define sprint goals • Select backlog items • Estimate effort • Assign tasks • Update schedule	Every 2 weeks Friday 16:00 - 17:00 (after retro)

Table 41: Planned Meetings Overview

8.2.2 Planning Tools

Tool	Use Case
GitHub	• Version control • GitHub Actions (CI/CD) • Pull requests • GitHub notifications via Discord
Typst	• Project documentation • PDF generation
Microsoft Teams	• File sharing • Communication
Jira	• Task management • Sprint planning • Backlog tracking • Issue tracking • Time tracking
Miro	• Brainstorming • Diagrams • Project planning
Discord	• Online meetings • Informal communication • Quick coordination • GitHub activity notifications

Table 42: Planning Tools

8.3 Schedule

The project follows the OST internal SCRUM+ framework for its overall organization. SCRUM+ combines elements of Scrum with the Rational Unified Process (RUP). Long-term planning is based on the RUP approach, while short-term development is organized using Scrum with two-week iterations.

At the start of each iteration, we estimate all tickets with storypoints. The story **GC-54 Make Player** is the reference story which is worth 5 storypoints.

8.3.1 Work Items

We work with 5 types of Work Items

Name	Description
Epic	A group of features that we want to implement.
Story	Describes a feature that is directly related to the game
Task	Everything that isn't directly related to the game
Bug	Bugs are created when a bug is found and fixed in an unplanned matter.
Documentation	Work related to the documentation

Table 43: Work Items

8.3.2 Long Term Plan

The Long Term Planning is separated into two categories:

- Milestones: Serve as checkpoint for releases and review meetings.
- Epics: Provide the structure to organize the stories.

8.3.2.1 Milestones

The milestones for the review meetings and for the releases are interleaved and independent.

In agile project management it is common to reevaluate the Long Term Plan during the project. For this reason there are two Milestone Tables. The first table shows the Initial Milestones that were defined at the beginning of the project and the second table contains updated Milestones as of 07.04.2026.

8.3.2.1.1 Initial Milestones

Nr	Milestone Name	Date	Description
M01	Review 1	10.03.2026	Initial Project Setup
M02	Review 2	24.03.2026	Requirements
M03	Review 3	14.04.2026	End of Elaboration
M04	Review 4	28.04.2026	Quality
M05	Review 5	12.05.2026	Architecture
M06	Project Presentation	12.06.2026	Final Presentation to SEProject-Advisors and fellow students. Completion of SEProject.
M07	Pre-Alpha Release	20.03.2026	First playable prototype is ready. Core gameplay loops are implemented. The player can chop trees, pick up wood and expand the GlowCore.
M08	Alpha Release	10.04.2026	MVP is done and working. Core gameplay Mechanics are implemented. Player can craft, build, expand the map and he has an inventory. The player can save his savestate.
M09	Beta Release	15.05.2026	More Biomes and Materials are added. There is a combat system with enemies and peaceful mobs. There is a title screen and a settings menu. Automation mechanics are implemented. There are more GlowCore Phases.
M10	Official Release and Final Submission	05.06.2026	The game is polished and has no mayor bugs. The game has a beginning and an ending.

Table 44: Initial Milestones

8.3.2.1.2 Updated Milestones as of 07.04.2026

Milestone M08 - Beta Release was updated and the combat system and the automation mechanics were removed. The reduction in scope allows the team to deliver a stable and more polished game at the end. The savestate system was moved from Alpha to Beta. We prioritize quality over quantity. If there is enough time at the end of the project, we can still implement the features as a bonus.

Nr	Milestone Name	Date	Description
M01	Review 1	10.03.2026	Initial Project Setup
M02	Review 2	24.03.2026	Requirements
M03	Review 3	14.04.2026	End of Elaboration
M04	Review 4	28.04.2026	Quality
M05	Review 5	12.05.2026	Architecture
M06	Project Presentation	12.06.2026	Final Presentation to SEProject-Advisors and fellow students. Completion of SEProject.
M07	Pre-Alpha Release	20.03.2026	First playable prototype is ready. Core gameplay loops are implemented. The player can chop trees, pick up wood and expand the GlowCore.
M08	Alpha Release	10.04.2026	MVP is done and working. Core gameplay Mechanics are implemented. Player can craft, build, expand the map and he has an inventory.
M09	Beta Release	15.05.2026	More Biomes and Materials are added. There is a title screen and a settings menu. There are more GlowCore Phases. The player can save his savestate.
M10	Official Release and Final Submission	05.06.2026	The game is polished and has no mayor bugs. The game has a beginning and an ending.

Table 45: Updated Milestones**8.3.2.2 Epics**

Epic Name	Description/Requirements
Documentation	This epic includes all documentation related work.
Project Setup	This epic contains all work that is done in the Inception-Phase.
Core Gameplay Loop Mechanics	- The player can chop down trees, collect wood and feed it into the GlowCore. - The GlowCore levels up and expands the map. - The player is restricted to the unlocked map. The grid system is implemented.
Inventory, Crafting, Building	- Inventory: The player has an inventory and can store a limited amount of items. He can have items in his hands and in his backpack(or similar). - Crafting: The player can craft items to other items on different workstations. - Building: The player can place objects onto the world-grid.
Graphics	- The Artstyle of the game ist set. - The 3D Models follow a consistent Artstyle. - A Shader is implementet.
Game Framework & Completeness	This epic contains everything that makes a game complete and feel like a game. - Intro: When the Game starts the GlowCore Logo is displayed. After the title screen a Cutscene is played that leads into the game. - Ending: A ending of the game is defined. When the player finishes the game, a Cutscene is played and the credits roll. - UI: A Title Screen is available. In-Game there is a Debug Screen, and a Settings Menu. - Gamedata Saving: The player can save his progress.
Big World & GlowCore Phases	- World building: The game has a complete world to play in. - GlowCore Phases: The GlowCore has multiple phases, all of which need different materials to level up. The GlowCore changes visually with every Level.
Biomes & Materials	- The world has multiple biomes: e.g. Forest, Grassland, Mountain, Volcano, Jungle etc. - Each biome has unique resources. - There are Caves which you can enter.
Combat	- Enemies can spawn and fight with the player. - The player can take damage and can die. - A respawn mechanic is set up. - There are peaceful mobs.
Resource Automation	- The Player can replant trees. - The Player can craft machines that automatically gather resources.
Playtesting & Release	- Playtesting: The game is tested by players that play it for the first time (blind test). - Release: The game is released on steam on its own steam page and has a trailer, screenshots and a description.

Table 46: Epics

The following image shows the Timeline in Jira as of 04.03.2026.

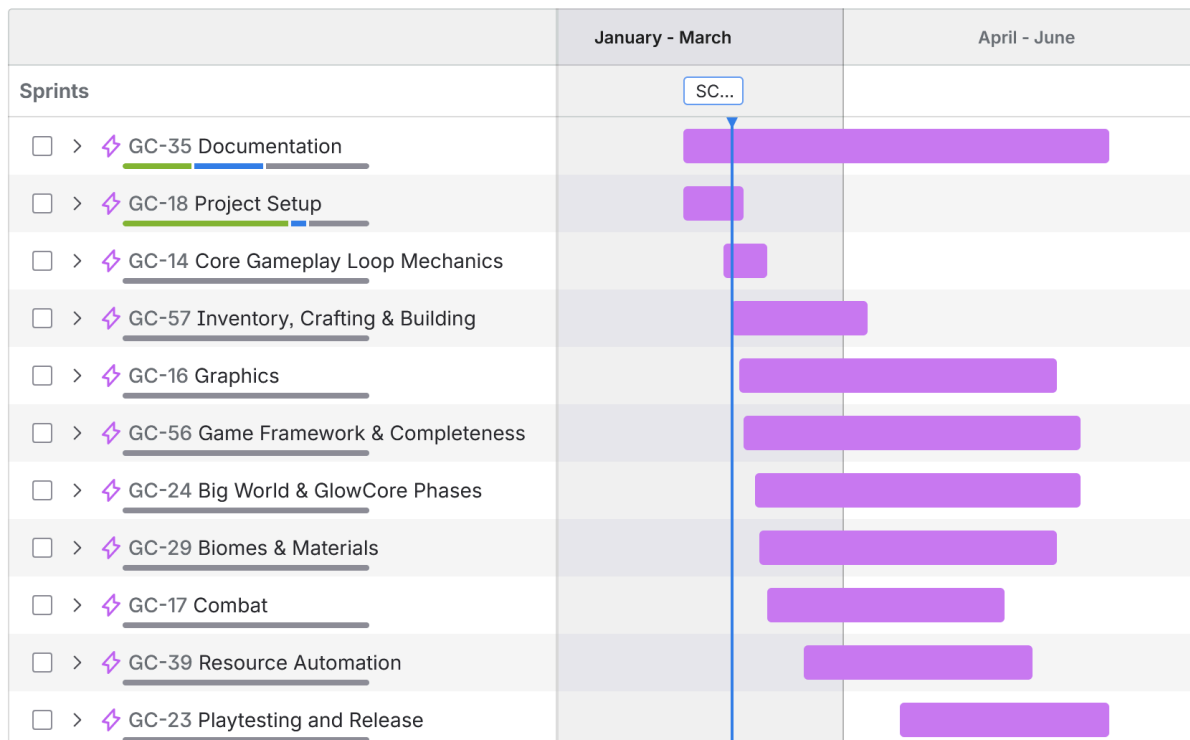


Image 10: Initial Jira Timeline

The Long Term Plan was updated on 07.04.2026. The following image shows the new Timeline in Jira as of 07.04.2026.

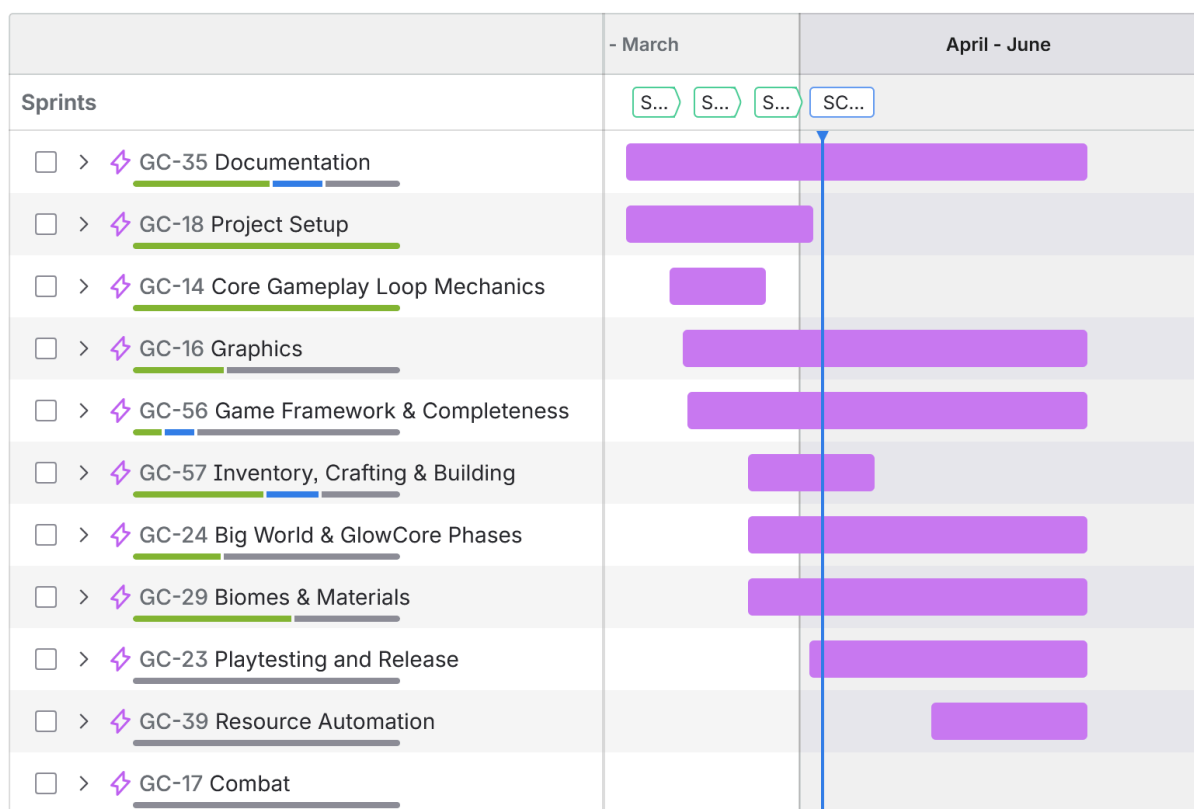


Image 11: Updated Jira Timeline

8.3.2.3 Initial Backlog & MVP

The priority of Stories is defined by the ordering in the backlog. As we progress in Agile, we may divide Stories further and add new Stories. The MVP is reached when every Story up to and including **GC-101 Build World** is completed.

The following is the Initial Backlog as of 04.03.2026.

▼

Backlog

(44 work items)

0

0

0

Create sprint

	GC-54	There is a player	CORE GAMEPLAY LO...	TO DO ▼	-	
	GC-78	Create Tree-Node 3D Model	GRAPHICS	TO DO ▼	-	
	GC-95	Create Wood-Item 3D Model	GRAPHICS	TO DO ▼	-	
	GC-47	Trees can be chopped	CORE GAMEPLAY LO...	TO DO ▼	-	
	GC-109	Make GlowCore Phase 1 3D Model	GRAPHICS	TO DO ▼	-	
	GC-46	GlowCore expands World	CORE GAMEPLAY LO...	TO DO ▼	-	
	GC-69	Make Inventory-System	INVENTORY, CRAFTI...	TO DO ▼	-	
	GC-74	Save Game-Data	GAME FRAMEWORK &...	TO DO ▼	-	
	GC-108	Create Axe 3D Model	GRAPHICS	TO DO ▼	-	
	GC-97	Create Coal 3D Model	GRAPHICS	TO DO ▼	-	
	GC-106	Add Coal Node and Item	BIOMES & MATERIALS	TO DO ▼	-	
	GC-96	Create Stone 3D Model	GRAPHICS	TO DO ▼	-	
	GC-93	Add Stone Node and Item	BIOMES & MATERIALS	TO DO ▼	-	
	GC-107	Make Pickaxe 3D Model	GRAPHICS	TO DO ▼	-	
	GC-102	Add Pickaxe	BIOMES & MATERIALS	TO DO ▼	-	
	GC-79	Add Grassland Biome	BIOMES & MATERIALS	TO DO ▼	-	
	GC-111	Make GlowCore Phase 2 3D Model	GRAPHICS	TO DO ▼	-	
	GC-100	Add GlowCore Phase 2	BIG WORLD & GLOWC...	TO DO ▼	-	
	GC-71	Make Crafting-System	INVENTORY, CRAFTI...	TO DO ▼	-	
	GC-70	Make Building-System	INVENTORY, CRAFTI...	TO DO ▼	-	
	GC-77	Implement Pixelize-Shader	GRAPHICS	TO DO ▼	-	
	GC-103	Add Chest	INVENTORY, CRAFTI...	TO DO ▼	-	
	GC-101	Build World	BIG WORLD & GLOWC...	TO DO ▼	-	
	GC-99	Add Rock Biome	BIOMES & MATERIALS	TO DO ▼	-	
	GC-72	Game Intro	GAME FRAMEWORK &...	TO DO ▼	-	

GC-73

Game Ending

GAME FRAMEWORK &...

TO DO ▼

-

GC-105

Add Combat Logic

COMBAT

TO DO ▼

-

GC-124

Create Enemy 3D Model

GRAPHICS

TO DO ▼

-

GC-81

Add Enemy

COMBAT

TO DO ▼

-

GC-75

Ingame Debug Settings

GAME FRAMEWORK &...

TO DO ▼

-

GC-76

Player Settings-Menu

GAME FRAMEWORK &...

TO DO ▼

-

GC-125

Create Peaceful-Mob 3D Model

GRAPHICS

TO DO ▼

-

GC-82

Add peaceful Mob

COMBAT

TO DO ▼

-

GC-123

Make Sword 3D Model

GRAPHICS

TO DO ▼

-

GC-83

Add Sword

COMBAT

TO DO ▼

-

GC-120

Make Axe-Machine 3D Model

GRAPHICS

TO DO ▼

-

GC-114

Add Axe-Machine

RESOURCE AUTOMAT...

TO DO ▼

-

GC-121

Make Replant-Machine 3D Model

GRAPHICS

TO DO ▼

-

GC-115

Add Replant-Machine

RESOURCE AUTOMAT...

TO DO ▼

-

GC-122

Make Rotater-Block 3D Model

GRAPHICS

TO DO ▼

-

GC-116

Add Rotater-Block

RESOURCE AUTOMAT...

TO DO ▼

-

GC-117

Conduct Playtesting with User

PLAYTESTING AND R...

TO DO ▼

-

GC-118

Release Game on Steam

PLAYTESTING AND R...

TO DO ▼

-

GC-119

Make Marketing Material for Steam Page

PLAYTESTING AND R...

TO DO ▼

-

Image 12: Initial Backlog

8.3.3 Short Term Plan

In a Sprint the priority of the tasks and stories is defined by the ordering.

8.3.3.1 Sprint 1 (Sprint Zero) 20.02.2026 - 06.03.2026

The following shows the Jira sprint backlog of the first sprint. Since this was Sprint Zero, not all guidelines were followed initially because they had not yet been established.

Work	Assignee	Status
 GG-88 Document Long and Short Term Plan	 Nathanael Fässler	DONE 
 GG-135 Document Sprint Planning Meeting Sprint 2	 Nathanael Fässler	DONE 
 GG-92 Setup Contract with OST about Ownership	 Nathanael Fässler	DONE 
 GG-89 Document Meeting Weekly Scrum Sprint 1	 Nathanael Fässler	DONE 
 GG-4 Concept for Code Quality, Linter etc.	 Yoris Kucera	DONE 
 GG-3 Unity Setup	 Yoris Kucera	DONE 
 GG-126 Documentation list of tables	 Cedric Cathomas	DONE 
 GG-1 Risk Specification	 Yoris Kucera	DONE 
 GG-91 Document Guidelines for code and documentation	 Yoris Kucera	DONE 
 GG-6 GitLab Timetracking Research	 Dominik Wyss	DONE 
 GG-5 GitLab Pipeline Setup	 Dominik Wyss	DONE 
 GG-2 GitHub Setup	 Yoris Kucera	DONE 
 GG-87 Document Meetings Structure	 Cedric Cathomas	DONE 
 GG-84 Typst documentation changes	 Cedric Cathomas	DONE 
 GG-86 Document Collaboration as a team	 Cedric Cathomas	DONE 
 GG-7 Typst Documentation Setup	 Cedric Cathomas	DONE 
 GG-8 People, Cost, Time Documentation	 Cedric Cathomas	DONE 
 GG-85 Roles Documentation	 Cedric Cathomas	DONE 
 GG-12 Longtermplan & Milestones	 Nathanael Fässler	DONE 
 GG-9 Jira Setup	 Nathanael Fässler	DONE 
 GG-112 Unity Folder Structure	 Nathanael Fässler	DONE 
 GG-11 Steamworks Registration	 Nathanael Fässler	DONE 
 GG-13 Shorttermplan for first sprint	 Nathanael Fässler	DONE 

Image 13: Sprint 1 Backlog

8.3.3.2 Sprint 2 06.03.2026 - 20.03.2026

In this sprint the first actual work on the game was carried out in Unity. The focus was to implement an initial working prototype.

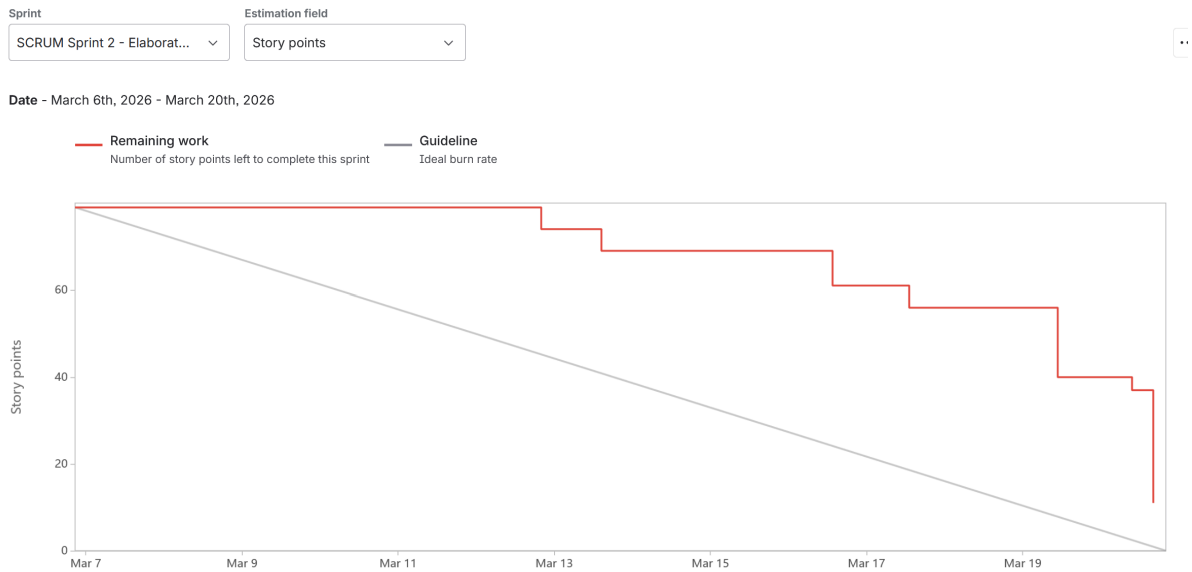


Image 14: Sprint 2 Burndown Chart

Report: SCRUM Sprint 2 - Elaboration 1

Scope changes log

Date	Key	Summary	Work type	Epic	Details of scope change	Change in estimation
2026-03-13	GC-138	GitHub Runner Disk Space usage to high	Bug		Work item added to sprint	-

Incomplete work items

Key	Summary	Work type	Epic	Status	Assignee	Story points
GC-130	Unity Testing Setup and configure Pipeline	Task	PROJECT SETUP	TESTING	NP	3
GC-133	Implement Basic inventory system	Story	INVENTORY, CRAFTING & BUIL...	IN PROGRESS	NP	8

Completed work items

Key	Summary	Work type	Epic	Status	Assignee	Story points
GC-85	Create Wood-Item 3D Model	Story	GRAPHICS	DONE	NP	1
GC-28	Create Tree-Node 3D Model	Story	GRAPHICS	DONE	NP	1
GC-34	Make Player	Story	CORE GAMEPLAY LOOP MECHA...	DONE	NP	5
GC-109	Make GlowCore Phase 1 3D Model	Story	GRAPHICS	DONE	NP	2
GC-47	Trees can be chopped	Story	CORE GAMEPLAY LOOP MECHA...	DONE	NP	5
GC-48	GlowCore expands World	Story	CORE GAMEPLAY LOOP MECHA...	DONE	NP	8
GC-108	Create Axe 3D Model	Story	GRAPHICS	DONE	NP	1
GC-106	Implement Node Hover Picking	Story	CORE GAMEPLAY LOOP MECHA...	DONE	NP	8
GC-130	Document Functional Requirements	Documentation	DOCUMENTATION	DONE	NP	8
GC-131	Document Non-Functional Requirements	Documentation	DOCUMENTATION	DONE	NP	8
GC-132	Document Domain-Model	Documentation	DOCUMENTATION	DONE	NP	13
GC-134	Pipeline for doc build	Task	PROJECT SETUP	DONE	NP	5
GC-136	Make UI Sketches	Task	GAME FRAMEWORK & COMPLET...	DONE	NP	3
GC-138	GitHub Runner Disk Space usage to high	Bug		DONE	NP	-

Image 15: Sprint 2 Backlog

8.3.3.3 Sprint 3 20.03.2026 - 03.04.2026

In Sprint 3 we focused on implementing the MVP. The planned amount of storypoints was too ambitious so we had to move some Tasks into Sprint 4. From this experience we gained a better understanding of our team's capacity.

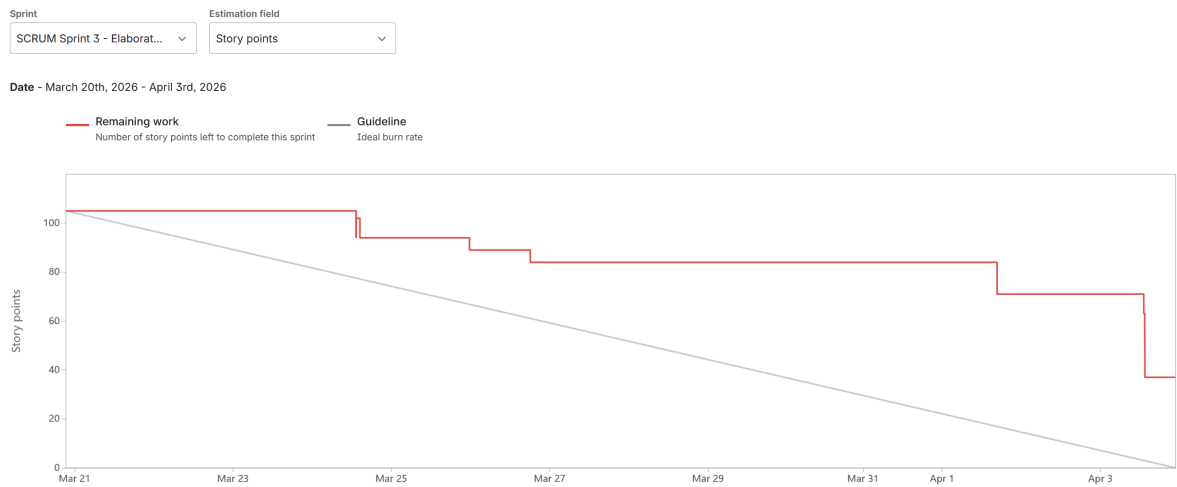


Image 16: Sprint 3 Burndown Chart

Report: SCRUM Sprint 3 - Elaboration 2

Scope changes log

Date	Key	Summary	Work type	Epic	Details of scope change	Change in estimation
2026-04-03	GC-198	Building doesn't work and pickaxe doesn't exist	Bug		Work item added to sprint	0

View in All work navigator

Incomplete work items

Key	Summary	Work type	Epic	Status	Assignee	Story points
GC-211	Make Crafting-System	Story	INVENTORY, CRAFTING & BUIL...	IN PROGRESS	JP	8
GC-244	Save Game-Data	Story	GAME FRAMEWORK & COMPLET...	IN PROGRESS	JP	13
GC-142	Make Crafting UI	Story	INVENTORY, CRAFTING & BUIL...	IN PROGRESS	JP	8
GC-148	Add GlowCore Upgrade UI	Story	INVENTORY, CRAFTING & BUIL...	TO DO	JP	8

View in All work navigator

Completed work items

Key	Summary	Work type	Epic	Status	Assignee	Story points
GC-352	Add Pickaxe	Story	ITEMS & MATERIALS	DONE	JP	3
GC-357	Create Coal 3D Model	Story	GRAPHICS	DONE	JP	1
GC-357	Make Player 3D Model	Story	GRAPHICS	DONE	JP	1
GC-292	Implement Building-System	Story	INVENTORY, CRAFTING & BUIL...	DONE	JP	13
GC-348	Make Inventory UI	Story	INVENTORY, CRAFTING & BUIL...	DONE	JP	13
GC-498	Make Inventory-System	Story	INVENTORY, CRAFTING & BUIL...	DONE	JP	8
GC-368	Create Stone 3D Model	Story	GRAPHICS	DONE	JP	1
GC-302	Make Pickaxe 3D Model	Story	GRAPHICS	DONE	JP	1
GC-308	Add Coal Node and Item	Story	ITEMS & MATERIALS	DONE	JP	1
GC-393	Add Stone Node and Item	Story	ITEMS & MATERIALS	DONE	JP	1
GC-372	Implement Pickaxe-Shader Prototype	Story	GRAPHICS	DONE	JP	8
GC-383	Build World for Alpha Release	Story	3D WORLD & GLSL/SHADER PR...	DONE	JP	3
GC-343	Implement Tool Logic	Story	INVENTORY, CRAFTING & BUIL...	DONE	JP	5
GC-348	Add Crafting Table 3D Model	Story	GRAPHICS	DONE	JP	1
GC-333	Implement Basic inventory system	Story	INVENTORY, CRAFTING & BUIL...	DONE	JP	8
GC-30	Unity-Testing Setup and configure Pipeline	Task	PROJECT SETUP	DONE	JP	3
GC-198	Building doesn't work and pickaxe doesn't exist	Bug		DONE	JP	0

View in All work navigator

Image 17: Sprint 3 Backlog

8.4 Risk Management

This chapter describes potential risks to the project and how we plan to mitigate them. We identified 8 risks, which are listed in the table below. Additionally, these risks are visualized in two risk matrices, one showing the initial risk assessment and the other showing the risk assessment after mitigation strategies are applied.

No.	Risk	Description	Mitigation Strategy
R01	Lack of familiarity with Unity	None of our team members have any significant experience with the Unity Engine. As such, this could heavily impact our development times.	We made ourselves familiar with the engine before the actual start of the project, to get an idea of the work-flows.
R02	Unity CI/CD Complexity	We are depending on Unity's build system being robust enough to work in the context of a CI/CD pipeline. We also potentially rely on pre-existing	We will set up the CI/CD pipeline very early into the project and did earlier tests. We tried out pre-existing

		GitHub Actions for Unity, which may not be well-maintained or compatible with our project.	GitHub Actions to make sure they satisfy our needs.
R03	Scope/Feature Creep	As we are developing a game, we run the risk of encountering scope creep, i.e. continuously planning new features, which would result in us eventually not being able to keep up with the workload.	We created a detailed project plan with long-term and short-term planning, along with well defined milestones and a clear MVP.
R04	Bad Code Quality	In large codebases it is easy to end up with badly designed systems that don't fit together with the rest of the code. This would hinder further development.	We do mandatory code reviews before changes can be merged. Additionally, we will set up CI/CD pipelines to automatically run code quality and formatting checks on pull requests.
R05	Team Member is MIA	A team member might be MIA for an extended period of time due to various reasons, such as an illness or damaged hardware. This would reduce our capacity and slow down development.	We can't reduce the possibility of this occurring, but we can distribute work across the remaining team members. We also avoid knowledge monopolies and always aim to share discoveries with the other team members.
R06	Personal Conflicts	There is always the potential for personal conflicts within the team, which would complicate development.	We openly communicate any interpersonal issues and try to resolve conflicts as early as possible.
R07	Data Loss	There is the potential of local storage media failing, resulting in significant data loss.	We mitigate this by frequently committing our changes to GitHub, and keeping untracked files in cloud based locations.
R08	Workload Underestimation	We might be underestimating the amount of work required for specific features and tasks.	We plan with a buffer and try to be conservative with our estimates. We also try to break down large tasks into smaller ones, which are easier to estimate.

Table 47: Risk Analysis

8.4.1 Risk Matrix

The below table shows the risk assessment of the identified risks, before any mitigation strategies are applied. The risks are categorized based on their probability of occurrence and their potential severity.

Probability	Severity			
	Negligible	Marginal	Critical	Catastrophic
Certain				
Likely			R04	R01
Possible		R02/R06		R03/R08
Unlikely				R05
Rare				R07
None				

Table 48: Risk Matrix

8.4.2 Risk Matrix (Mitigated)

The below table shows the risk assessment of the identified risks, after mitigation strategies have been applied.

Probability	Severity			
	Negligible	Marginal	Critical	Catastrophic
Certain				
Likely				
Possible				
Unlikely		R06	R01/R04/R05	R08
Rare	R07	R02		
None				R03

Table 49: Risk Matrix (Mitigated)

8.4.3 Assessed Risk Matrix (14.04.2026)

The below table shows the risk assessment of the identified risks, after mitigation strategies have been applied.

Probability	Severity			
	Negligible	Marginal	Critical	Catastrophic
Certain				
Likely				
Possible			R01	
Unlikely		R06/R02	R04/R05	R08
Rare	R07			
None				R03

Table 50: Assessed Risk Matrix (14.04.2026)

8.4.4 Adjustment of Current Risk Assessment

In the current risk matrix, we increased the probability by one level for both R01 (Lack of familiarity with Unity) and R02 (Unity CI/CD Complexity).

For R01, we observed that working with Unity involves more aspects than initially expected, especially issues that only become visible during actual implementation. The project description already pointed out that Unity could be a risk, so we were aware of this from the beginning. However, during development we realized that some parts are more complex in practice. Despite this, we are still very motivated to use Unity for our project and try to reduce the risk through clear communication and well-defined implementation strategies.

For R02, there are recurring challenges with the CI/CD setup in Unity, which is why we increased the probability here as well. In particular, there is a risk of running out of GitHub Actions minutes. At the moment, we expect the available minutes to be sufficient, but if needed, we already have a plan to set up and use a self-hosted runner on our own machine.

Overall, both risks are now considered slightly more likely, but appropriate mitigation measures are in place.

9 Time Tracking Report

9.1 Time Tracking

Every member tracks time on Jira on the corresponding task/story. Time spent in meetings is documented in the meeting protocols. The time is logged in 15 minute blocks in the format e.g. 3h 15m.

9.2 Time Statistics

The time is grouped by member and by sprint. Per sprint and person the time is rounded to the nearest hour.

Sprint	Cedric Cathomas	Nathanael Fässler	Yoris Kucera	Dominik Wyss	Total
1	23h	29h	21h	24h	97h
2	22h	20h	19h	23h	84h
3	18h	13h	19h	15h	65h
4					
5					
6					
7					
Total	63h	62h	59h	62h	246h

Table 51: Time Statistics Sprint 1

10 Personal Reports

11 Meeting Minutes

11.1 Weekly Scrum / 28.02.2026

Field	Value
Date	28.02.2026
Time	11:00–14:00
Sprint	Sprint 1
Location	Online
Participants	Nathanael Fässler, Dominik Wyss, Yoris Kucera
Excused	Cedric Cathomas
Absent	-
Meeting lead	Yoris Kucera
Minutes by	Nathanael Fässler
Links	-

11.1.1 Agenda

Agenda item	Owner	Result / Decision
GitHub Setup	Yoris	- We switched from GitLab to GitHub The following rules were discussed: - Pull Request requires one approval that is not the same person as the author. - Merging Methods are Merge and Squash. - Every Jira Task corresponds to one branch.
CICD Pipeline Setup	Dominik	- Web build and .exe build succeeded. - Pages Architecture with main and dev branches decided
Timetracking	Dominik	- We won't use GitLab timetracking because we switched to GitHub - Timetracking is in Jira: Everyone tracks time for individual Tasks/Stories.
Unity/Coding Setup	Yoris	Coding Guidelines decided
Steam Release	Nathanael	- Registered on Steamworks - Up to 30 days waiting time until application is processed - Maybe we need to make a contract with OST.
Testing	Nathanael	Unit-Test and Integration-Test Strategy discussed.
Long/Shorttermplan	Nathanael	- Hierarchy: There are Epics which contain Stories. - Everything that is not a game-feature is a task. - Longtermplan with epics according to the Jira Timeline - Shorttermplan per sprint on the Jira board

		• Priority is defined by the order in the backlog in Jira • MVP is defined • We will estimate Tasks/Stories with Story-points • On Jira we can create Bug-Tickets which we will not estimate with Storypoints
--	--	---

11.1.2 Todos

Todo	Owner	Due
Finish CICD Pipeline	Dominik	06.03.2026
Ask Advisor about contract	Nathanael	04.03.2026
Ask Advisor about project plan	Nathanael	04.03.2026

11.2 Weekly Scrum and Sprint Retrospective / 06.03.2026

Field	Value
Date	06.03.2026
Time	14:00-16:00
Sprint	Sprint 1
Location	Online
Participants	Nathanael Fässler, Dominik Wyss, Yoris Kucera, Cedric Cathomas
Excused	-
Absent	-
Meeting lead	Nathanael Fässler
Minutes by	Cedric Cathomas
Links	-

11.2.1 Agenda

Agenda item	Owner	Result / Decision
Board Review	All	- Walked through all current board tasks - Updated status on completed tasks
Done Tasks and New Functionalities	All	- Reviewed completed work from Sprint 1 - Discussed new functionalities and rules
Open Questions	All	Discussed open questions regarding project setup and workflow
Unity Graphics in Repository	Nathanael Fässler	Only .fbx files are to be committed to the repository for 3D graphics in Unity
Documentation Versioning	Cedric Cathomas	- Tags for documentation versions start with "d" (e.g. d1.1) - Version is updated automatically on pull request to main via GitHub Action
How do we manage smaller tasks that are not worth creating a new task on jira. (small documentation changes)	Dominik Wyss	We create ticket for small backlog tasks that do not need an own Jira item so we can reference it on our pull request and use the name of that ticket for the branch
Decide if we need a separate pdf for protocols	Cedric Cathomas	We decided that there's no use in having a separate page just for meeting minutes, so we have one thing less to worry about

11.2.2 Sprint 1 Retrospective

11.2.2.1 What Went Well

- Good and fast communication
- Good planning on Jira
- Good preplanning visible
- Product Owner did a good job
- Good and understandable selection of planning and working tools
- Pipeline and GitHub are working
- Typst is set up
- General setup went very well

11.2.2.2 What Didn't Go Well

- Problems with GitLab
- Didn't have enough time for some tasks

11.2.2.3 Improvements

- Better time management
- Better definitions of done
- Use GitHub from the beginning

11.2.3 Todos

Todo	Owner	Due
Create ticket for small backlog tasks that do not need an own Jira item	Nathanael Fässler	07.03.2026
Create all table references so documentation looks nice and page breaks are clean	Cedric Cathomas	07.03.2026
Adapt milestone dates to single dates (e.g. 16.02.2026 instead of a time frame)	Cedric Cathomas	07.03.2026
clean up documentation (spaces that are too big)	Cedric Cathomas	07.03.2026
Change date format in documentation to use dots	Cedric Cathomas	07.03.2026
Create task for combined build and documentation pipeline	Nathanael Fässler	07.03.2026
Use figures for images so they appear in the list of figures	Nathanael Fässler	07.03.2026
remove seperate meeting minutes pdf typst code	Cedric Cathomas	07.03.2026

11.3 Sprint Planning / 06.03.2026

Field	Value
Date	06.03.2026
Time	16:00–18:00
Sprint	Sprint 2
Location	Online
Participants	Nathanael Fässler, Dominik Wyss, Yoris Kucera, Cedric Cathomas
Excused	-
Absent	-
Meeting lead	Yoris Kucera
Minutes by	Nathanael Fässler
Links	-

11.3.1 Sprint goal

- Build first working prototype.
- Define Functional Requirements and Non-Functional Requirements.
- Create Domain Model

11.4 Sprint Review (Advisor) / 10.03.2026

Field	Value
Date	10.03.2026
Time	15:00–16:00
Sprint	Sprint 2
Location	Building 8
Participants	Thomas Bocek, Dominik Wyss, Yoris Kucera, Nathanael Fässler, Cedric Cathomas
Excused	-
Absent	-
Meeting lead	Yoris Kucera
Minutes by	Nathanael Fässler
Links	-

11.4.1 Agenda

Agenda item	Owner	Result / Decision
Status of Work	Yoris	Feedback was good
Project Plan	Yoris	Good impression. Advisor liked that we asked him at 02.03.2026 on our initiative
Showcase Pipeline	Dominik	Positive that Dominik sent the pipeline beforehand
Feedback	Thomas	Overall very good
Questions about Grading & Review Schedule	Dominik & Nathanael	- To get a better grade we have to do more than what is required on the checklist. - The group can choose when the Review Meetings take place. - Meeting 5 has to be before 05.06.2026 (final submission)

11.4.2 Todos

Todo	Owner	Due
Update README on GitLab to indicate that project has moved to GitHub	Yoris	20.03.2026
Configure Pipeline to build documentation	Not assigned	20.03.2026
Add AGENTS.md	Cedric Cathomas	20.03.2026

11.4.3 Next dates

Next date	When
Review 2 - Requirements	24.03.2026 15:00

11.5 Weekly Scrum / 13.03.2026

Field	Value
Date	13.03.2026
Time	14:30-15:30
Sprint	Sprint 2
Location	Online
Participants	Cedric Cathomas, Nathanael Fässler, Dominik Wyss, Yoris Kucera
Excused	-
Absent	-
Meeting lead	Dominik Wyss
Minutes by	Yoris Kucera
Links	-

11.5.1 Agenda

Agenda item	Owner	Result / Decision
Time Tracking	Dominik Wyss	- New time tracking report system integrated in Jira (Worklogs Report) - Task does not need to be assigned for time to be tracked
Documentation Build Pipeline	Yoris Kucera	- Pipeline for building documentation on GitHub Actions is set up and working - Automatically deployed to pages on push to dev and main branches.
Domain Model	Yoris Kucera	- First version of domain model created - State machine for enemy created
Functional Requirements	Nathanael Fässler	- Documented Functional Requirements, Actors and Personas - Documented Use Cases - Use Cases formatting could be improved
Player and Camera	Nathanael Fässler	- Player controller implemented and working, controls with WASD - Camera implemented and working, controls with mouse movement - Camera follows player and rotates around it, fixed height and distance
Non-Functional Requirements	Cedric Cathomas	- Documented Non-Functional Requirements - Priority Matrix for NFRs - NFRs grouped by category and numbered (e.g. NFR101, NFR102, NFR403, etc.)
GitHub CI/CD Failing	Dominik Wyss/Yoris Kucera	- Pipeline fails because of lack of disk space on the runner - Possible solutions: ▸ Clean up even more storage ▸ Make repository public ▸ Use self-hosted runner with more storage ▸ Rent server at OST

UI Sketches	Dominik Wyss/ Cedric Cathomas	• Sketches for Main Menu ▸ Scene from the game shown in the background ▸ Buttons for New Game, Continue, Save Files, Settings, Exit • Sketch for New Game menu ▸ Give name to world • Sketch for Save File Menu ▸ List of save files with name and date ▸ Button to delete save file ▸ Show glowcore level? ▸ Separate list for auto saves and manual saves? • Sketch for Settings Menu ▸ Sliders for music and sound effects volume ▸ Slider for mouse sensitivity? ▸ Graphics options? • Sketch for Pause Menu
-------------	----------------------------------	--

11.5.2 Todos

Todo	Owner	Due
Fix WebGL Build Pipeline	Yoris Kucera	20.03.2026
Unify NFRs, FRs and Domain Model	Yoris Kucera	20.03.2026
Adjust README on GitLab	Dominik Wyss	20.03.2026
Agents.md	Cedric Cathomas	20.03.2026
Explicitly address review feedback in Teams	Dominik Wyss	20.03.2026

11.6 Weekly Scrum and Sprint Retrospective / 20.03.2026

Field	Value
Date	20.03.2026
Time	13:00–15:00
Sprint	Sprint 2
Location	Online
Participants	Nathanael Fässler, Dominik Wyss, Yoris Kucera, Cedric Cathomas
Excused	-
Absent	-
Meeting lead	Yoris Kucera
Minutes by	Nathanael Fässler
Links	-

11.6.1 Agenda

Agenda item	Owner	Result / Decision
Planned Inventory System Architecture	Yoris	- ScriptableObject Item represents InventoryItem and WorldItem at the same time. - ItemStack ScriptableObject hold Item and amount.
World Grid & GlowCore expansion	Cedric	- Showed simple prototype for tree generation on border expansion. - Grid represented by 2D Array.
Node Interaction System	Dominik	- Setting for Interaction Range for every Node. - Problem with InputSystem to track how long a button is held.

11.6.2 What went well

- Had fun beginning the work in Unity.
- Good cooperation: Questions by team-members are answered quickly.
- The planning starts to pay off.

11.6.3 What didn't

- Unclear dependencies of tasks.
- CI/CD work can be annoying.

11.6.4 Improvements

- Keep Jira Board up to date.
- Better coordinate the task assignment.

11.6.5 Todos

Todo	Owner	Due
------	-------	-----

Send documentation to Advisor	Dominik	21.03.2026
Make Meeting Minutes	Nathanael	23.03.2026
Finish and merge prototype	Yoris	23.03.2026

11.7 Sprint Planning / 20.03.2026

Field	Value
Date	20.03.2026
Time	15:00–16:00
Sprint	Sprint 3
Location	Online
Participants	Nathanael Fässler, Dominik Wyss, Yoris Kucera, Cedric Cathomas
Excused	-
Absent	-
Meeting lead	Yoris Kucera
Minutes by	Nathanael Fässler
Links	-

11.7.1 Sprint goal

- Create working MVP.
- Implement the foundational Item and Node Architecture.

11.8 Sprint Review (Advisor) / 24.03.2026

Field	Value
Date	24.03.2026
Time	15:00–16:00
Sprint	Sprint 3
Location	Building 8
Participants	Thomas Bocek, Dominik Wyss, Yoris Kucera, Nathanael Fässler, Cedric Cathomas
Excused	-
Absent	-
Meeting lead	Yoris Kucera
Minutes by	Cedric Cathomas
Links	GitHub repository + Jira board (internal team links)

11.8.1 Agenda

Agenda item	Owner	Result / Decision
Documentation status (FR, architecture, protocols)	Yoris Kucera	Overall quality was positively received; minor consistency updates required (see Todos)
Current game state demo	Yoris Kucera	Game is playable as planned for pre-alpha release
Mockups	Yoris Kucera	Gave a clear picture of how our UI will look
Code quality (linter/status)	Yoris Kucera	Everything was good
Advisor feedback	Thomas Bocek	Very good feedback, overall impression better than last week

11.8.2 Todos

Todo	Owner	Due
FR: align color definitions and structure as in the NFRs	Cedric Cathomas	27.03.2026
Enemy state machine: add explicit color legend to diagram	Yoris Kucera	27.03.2026

11.8.3 Next dates

Next date	When
Review 3 - End of Elaboration	14.04.2026 15:00

11.9 Weekly Scrum / 27.03.2026

Field	Value
Date	27.03.2026
Time	14:00–14:45
Sprint	Sprint 3
Location	Online
Participants	Cedric Cathomas, Nathanael Fässler, Dominik Wyss, Yoris Kucera
Excused	-
Absent	-
Meeting lead	Dominik Wyss
Minutes by	Nathanael Fässler
Links	-

11.9.1 Agenda

Agenda item	Owner	Result / Decision
New Nodes and Items created with new 3D Models	Dominik Wyss	Scriptable Object Architecture works
Pixelate Shader	Yoris Kucera	- Team thinks it looks good - Showcased parameters of the shader - Camera angle and zoom needs refining
Inventory UI	Cedric Cathomas	Prototype is working
Node and Tile hovering Architecture	Nathanael Fässler	Will be implemented with events

11.9.2 Todos

Todo	Owner	Due
Explicitly address review feedback in Teams	Dominik Wyss	03.04.2026

11.10 Weekly Scrum and Sprint Retrospective / 03.04.2026

Field	Value
Date	03.04.2026
Time	13:00–14:00
Sprint	Sprint 3
Location	Online
Participants	Nathanael Fässler, Dominik Wyss, Yoris Kucera, Cedric Cathomas
Excused	-
Absent	-
Meeting lead	Cedric Cathomas
Minutes by	Dominik Wyss
Links	-

11.10.1 Agenda

Agenda item	Owner	Result / Decision
Add new nodes, items, tools	Dominik Wyss	- New ScriptableObjects and Prefabs - Updated MainWorld to demo new features
Pixelized Shader	Yoris Kucera	Working shader with editable fields
Building System	Nathanael Fässler	- Chest as first building block - Valid/invalid placement indicator
Inventory UI with system architecture changes	Cedric Cathomas	Inventory and handslot
Feedback Review 2: FR coloring indicator same as NFR	Cedric Cathomas	
Feedback Review 2: Enemy state machine: add explicit color legend to diagram	Yoris Kucera	

11.10.2 What went well

- Good communication within the team
- Strong interest and engagement in the project
- Attention to detail
- Review comments were questioned and discussed, not just - blindly accepted
- Everything came together into a working game

11.10.3 What didn't

- Conflicting ideas about how to implement features and resulting dependencies sometimes disrupted the workflow.
- Tickets should be described more clearly to support - conceptual implementation
- Planned too much
- The 3D player model was more difficult than expected

11.10.4 Todos

Todo	Owner	Due
------	-------	-----

Send message to advisor about feedback from last review	Dominik	10.04.2026
Send message to advisor about review 3, point A5. Remove A5 from checklist because we don't have external systems.	Dominik	10.04.2026

11.11 Sprint Planning / 03.04.2026

Field	Value
Date	03.04.2026
Time	14:00–15:30
Sprint	Sprint 4
Location	Online
Participants	Nathanael Fässler, Dominik Wyss, Yoris Kucera, Cedric Cathomas
Excused	-
Absent	-
Meeting lead	Cedric Cathomas
Minutes by	Dominik Wyss
Links	-

11.11.1 Sprint goal

- Finish tickets from old sprint
- Working prototype with all core mechanics
- Focus on diagramm and documentation changes for review 3

Bibliography

List of Figures

Image 1	Use Case Diagram	6
Image 2	Domain Model Diagram	19
Image 3	Enemy State Machine Diagram	20
Image 4	C4 System Context Diagram	21
Image 5	C4 Container Diagram	22
Image 6	C4 Component Diagram - Gameplay Logic	23
Image 7	C4 Component Diagram - Presentation Layer	24
Image 8	Interface Overview Diagram	25
Image 9	Savefile Structure	32
Image 10	Initial Jira Timeline	42
Image 11	Updated Jira Timeline	42
Image 12	Initial Backlog	43
Image 13	Sprint 1 Backlog	44
Image 14	Sprint 2 Burndown Chart	45
Image 15	Sprint 2 Backlog	45
Image 16	Sprint 3 Burndown Chart	46
Image 17	Sprint 3 Backlog	46

List of Tables

Table 1	Glossary	4
Table 2	UC Status Color Legend	7
Table 3	UC Overview – All Use Cases at a Glance	7
Table 4	UC01 - Install Game	8
Table 5	UC02 - Save Game	8
Table 6	UC03 - Break Nodes	8
Table 7	UC04 - Expand Map	8
Table 8	UC05 - Store Items	8
Table 9	UC06 - Craft Items	9
Table 10	UC07 - Build Nodes	9
Table 11	UC08 - Experience Story	9
Table 12	UC09 - Fight Enemies	9
Table 13	UC10 - Automate Resource Gathering	10
Table 14	UC11 - Change Settings	10
Table 15	NFR Status Color Legend	11
Table 16	NFR Overview – All Non-Functional Requirements at a Glance	11
Table 17	NFR Priority Matrix – MVP Relevance vs. Importance	12
Table 18	NFR101 – World Load Time	12
Table 19	NFR102 – In-Game Transition Time	13
Table 20	NFR103 – Frame Rate	13
Table 21	NFR104 – Save File Size	13
Table 22	NFR105 – Minimal Hardware Requirements	13
Table 23	NFR201 – Player Restriction Visibility	14
Table 24	NFR202 – Light Upgrade Progress Indication	14
Table 25	NFR203 – New Player Learnability	14
Table 26	NFR204 – Input Support	15

Table 27 NFR205 – Game Language	15
Table 28 NFR206 – Visual Style Consistency	15
Table 29 NFR301 – Save File Portability	16
Table 30 NFR302 – Game Progress Persistence	16
Table 31 NFR303 – Save Data Integrity	16
Table 32 NFR401 – Extensibility of Game Systems	17
Table 33 NFR402 – Automated Test Coverage	17
Table 34 NFR501 – Multi-Platform Support	17
Table 35 NFR502 – Engine and Rendering Pipeline	18
Table 36 NFR503 – Display Resolution Support	18
Table 37 Interface Contracts	26
Table 38 Team availability for review meetings	34
Table 39 Project Management Roles	36
Table 40 Developer Roles	37
Table 41 Planned Meetings Overview	38
Table 42 Planning Tools	38
Table 43 Work Items	39
Table 44 Initial Milestones	40
Table 45 Updated Milestones	40
Table 46 Epics	41
Table 47 Risk Analysis	47
Table 48 Risk Matrix	47
Table 49 Risk Matrix (Mitigated)	48
Table 50 Assessed Risk Matrix (14.04.2026)	48
Table 51 Time Statistics Sprint 1	49

Code Listings